EduForge: BKT-GenUI — Complete Agent Build Brief

Neuro-Symbolic Intelligent Tutoring System with Bayesian Knowledge Tracing + Generative UI

Version: 1.0 | **Date:** April 2026 | **Target:** National-Level Project Competition
Build Window: 5–6 days with GitHub Copilot Pro+ Agent
Infrastructure: Google Cloud Platform (GCP) with pre-configured gcloud CLI

Table of Contents

1. Project Vision & Research Grounding
2. System Architecture Overview
3. GCP Infrastructure Setup
4. Production Directory Structure
5. Feature Specifications — Teacher Portal
6. Feature Specifications — Async Processing Pipeline
7. Feature Specifications — BKT Engine
8. Feature Specifications — Student Learning Experience
9. Feature Specifications — GenUI Layer
10. Feature Specifications — Analytics Dashboards
11. Data Models & Firestore Schema
12. API Contracts
13. Frontend Architecture
14. Backend Services Architecture
15. Authentication & Security
16. Testing Strategy
17. Deployment Scripts & CI/CD
18. Day-by-Day Build Timeline
19. Competition Demo Script
20. pyBKT + GenUI Deep Integration Guide
21. VS Code Agent Skills & Tooling
22. Structured Logging & Chalk Strategy

---

1. Project Vision & Research Grounding

1.1 Core Innovation

EduForge is the first system to combine:

1. **Bayesian Knowledge Tracing (BKT)** — a probabilistic symbolic model that tracks each student's per-concept mastery over time
2. **Generative UI (GenUI)** — LLM-driven dynamic UI component generation based on context
3. **Pedagogical Constraint Layer** — BKT posterior as a *hard constraint* on what UI the LLM is allowed to generate

This is the **neuro-symbolic fusion**: the symbolic layer (BKT) constrains what the neural layer (LLM/GenUI) can produce. No existing EdTech product or GenUI library does this.

1.2 Research Citations (for competition submission)

Research	Key Finding	Applied In EduForge
Google GenUI Paper (Leviathan et al., 2025)	"LLMs are effective UI generators; custom UIs overwhelmingly preferred over markdown"	Student GenUI panel architecture
pyBKT (Badrinath et al., EDM 2021)	"150–600x speedup vs pure Python; tracks P(mastery), P(learn), P(slip), P(guess)"	BKT engine core (pyBKT library)
OpenUI by thesysdev (2025)	"67% fewer tokens than JSON-render; controlled component catalog per LLM call"	GenUI renderer with BKT-gated catalogs
CopilotKit AG-UI Protocol (2025)	"3 GenUI patterns: controlled, declarative, open-ended; shared agent-UI state"	Architecture reference for component patterns
Google A2UI (2025)	"Declarative JSON catalog; agent can only use pre-approved components"	Security-first catalog design
Google Document AI (GA 2025)	"Layout parser supports PPTX natively; Gemini layout parser for structure extraction"	Teacher PPT ingestion pipeline

1.3 The Gap This Fills

Every existing GenUI system (Vercel AI SDK streamUI, OpenUI, CopilotKit, Claude Artifacts, Google GenUI) answers: *"Given user intent, what UI should appear?"*

EduForge answers a different question: *"Given this student's BKT posterior across 8 concepts (P(mastery)= [0.28, 0.71, 0.45...]), what UI is pedagogically appropriate to render right now?"*

The BKT state is a **first-class input** to the UI generation pipeline — not a post-hoc filter.

---

2. System Architecture Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│                          TEACHER PORTAL                              │
│  Upload PPTs/PDFs → Create Lesson → Publish → View Analytics         │
└──────────────────────────────────────────────────────────────────────┘
                              │
                              │ HTTP POST (lesson + files)
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                    API GATEWAY (Cloud Run — api-service)             │
│  FastAPI • Firebase Auth JWT validation • Rate limiting              │
└──────────────────────────────────────────────────────────────────────┘
           │
           │ Pub/Sub publish
           ▼
┌──────────────────────────────────────────────────────────────────────┐
│              INGESTION PIPELINE (Cloud Run Job — ingestion-job)      │
│                                                                      │
│  1. Download PPT from Cloud Storage                                  │
│  2. python-pptx → extract slides, text, images                       │
│  3. Cloud Document AI (Layout Parser) → structured content           │
│  4. Gemini API → topic hierarchy + MCQ generation + concept graph    │
│  5. Store in Firestore: lessons/{id}/subtopics, mcqs, concepts       │
│  6. Notify via Pub/Sub → Cloud Run push → update lesson status       │
└──────────────────────────────────────────────────────────────────────┘
           │ Firestore writes
           ▼
┌──────────────────────────────────────────────────────────────────────┐
│                    FIRESTORE (Primary Database)                      │
│  lessons • subtopics • mcqs • students • bkt_states • sessions       │
│  Real-time listeners → student dashboard updates live                │
└──────────────────────────────────────────────────────────────────────┘
           │                          │
           │                          │ onSnapshot
           ▼                          ▼
┌──────────────────┐         ┌──────────────────────────────────────┐
│  BKT SERVICE     │         │           NEXT.JS FRONTEND           │
│  (Cloud Run)     │◄──────► │  Teacher Portal  │  Student Experience │
│                  │         │  Dashboard       │  GenUI Panel        │
│  pyBKT engine    │         │  Lesson Builder  │  Adaptive MCQ       │
│  Scaffold resolve│         │  Analytics HUD   │  AI Tutor Chat      │
│  Concept gating  │         │  Class Heatmap   │  Mastery HUD        │
└──────────────────┘         └──────────────────────────────────────┘
           │                          │
           └──────────────────────────► │ GenUI API call (with BKT state)
                                      ▼
                          ┌──────────────────────────┐
                          │  GENUI SERVICE (Cloud Run)│
                          │                          │
                          │  BKT state → scaffold lvl│
                          │  → allowed components    │
                          │  → OpenUI system prompt  │
                          │  → Claude API stream     │
                          │  → React component tree  │
                          └──────────────────────────┘
```

2.1 GCP Services Used

Service	Purpose	Why Chosen
Cloud Run	All backend services (stateless, containerized)	Auto-scales to zero, pay-per-use, supports long-running jobs
Cloud Run Jobs	Async PPT ingestion (up to 60 min)	No HTTP timeout limit unlike Cloud Run services
Cloud Pub/Sub	Trigger ingestion, notify completion	Decoupled async, retry logic built-in, push to Cloud Run
Cloud Storage (GCS)	PPT/PDF/image file storage	Integrated with Document AI, cheap egress, signed URLs
Cloud Document AI	PPTX layout parsing	Native PPTX support in GA (2025), Gemini-powered layout parser
Firestore	Primary database + real-time listeners	Real-time onSnapshot, scales horizontally, native Firebase Auth integration
Firebase Authentication	User auth (teacher/student roles)	Free tier generous, JWT works seamlessly with Firestore security rules
Cloud Tasks	Scheduled retries for failed ingestion	Exactly-once delivery guarantees
Artifact Registry	Docker image storage	Required for Cloud Run deployments
Secret Manager	API keys (Gemini, Claude, etc.)	Security best practice, accessible from Cloud Run via env vars
Cloud Logging + Monitoring	Observability	Automatic with Cloud Run; dashboards for competition demo
Firebase Hosting	Next.js frontend deployment	Free SSL, CDN, integrates with Firebase Auth

---

3. GCP Infrastructure Setup

> **Run all scripts in order. These are idempotent — safe to re-run.**

3.1 Prerequisites

```
# Verify gcloud is configured
gcloud auth list
gcloud config get-value project

# Set your project (replace with your actual project ID)
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1"
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")

echo "Project: $PROJECT_ID"
echo "Region: $REGION"
echo "Number: $PROJECT_NUMBER"
```

3.2 Enable All Required APIs

```
#!/bin/bash
# scripts/gcp/01_enable_apis.sh

set -e
PROJECT_ID=$(gcloud config get-value project)

echo "Enabling APIs for project: $PROJECT_ID"

gcloud services enable \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  cloudtasks.googleapis.com \
  pubsub.googleapis.com \
  storage.googleapis.com \
  documentai.googleapis.com \
  firestore.googleapis.com \
  firebase.googleapis.com \
  secretmanager.googleapis.com \
  artifactregistry.googleapis.com \
  logging.googleapis.com \
  monitoring.googleapis.com \
  aiplatform.googleapis.com \
  iam.googleapis.com \
  --project=$PROJECT_ID

echo "✅ All APIs enabled"
```

3.3 Create Artifact Registry

```
#!/bin/bash
# scripts/gcp/02_create_registry.sh

set -e
export REGION="us-central1"
export PROJECT_ID=$(gcloud config get-value project)

gcloud artifacts repositories create eduforge \
  --repository-format=docker \
  --location=$REGION \
  --description="EduForge container images" \
  --project=$PROJECT_ID

# Configure Docker to authenticate with the registry
gcloud auth configure-docker ${REGION}-docker.pkg.dev

echo "✅ Artifact Registry created: ${REGION}-docker.pkg.dev/${PROJECT_ID}/eduforge"
```

3.4 Create Service Accounts

```bash
#!/bin/bash
# scripts/gcp/03_create_service_accounts.sh

set -e
PROJECT_ID=$(gcloud config get-value project)
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")

# API Service Account
gcloud iam service-accounts create eduforge-api \
  --display-name="EduForge API Service" \
  --project=$PROJECT_ID

# Ingestion Job Service Account
gcloud iam service-accounts create eduforge-ingestion \
  --display-name="EduForge Ingestion Job" \
  --project=$PROJECT_ID

# BKT Service Account
gcloud iam service-accounts create eduforge-bkt \
  --display-name="EduForge BKT Service" \
  --project=$PROJECT_ID

# GenUI Service Account
gcloud iam service-accounts create eduforge-genui \
  --display-name="EduForge GenUI Service" \
  --project=$PROJECT_ID

API_SA="eduforge-api@${PROJECT_ID}.iam.gserviceaccount.com"
INGESTION_SA="eduforge-ingestion@${PROJECT_ID}.iam.gserviceaccount.com"
BKT_SA="eduforge-bkt@${PROJECT_ID}.iam.gserviceaccount.com"
GENUI_SA="eduforge-genui@${PROJECT_ID}.iam.gserviceaccount.com"

# Grant roles to API service account
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${API_SA}" \
  --role="roles/datastore.user"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${API_SA}" \
  --role="roles/pubsub.publisher"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${API_SA}" \
  --role="roles/storage.objectCreator"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${API_SA}" \
  --role="roles/secretmanager.secretAccessor"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${API_SA}" \
  --role="roles/cloudtasks.enqueuer"

# Grant roles to Ingestion service account
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${INGESTION_SA}" \
  --role="roles/datastore.user"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${INGESTION_SA}" \
  --role="roles/storage.objectAdmin"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${INGESTION_SA}" \
  --role="roles/documentai.apiUser"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${INGESTION_SA}" \
  --role="roles/aiplatform.user"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${INGESTION_SA}" \
  --role="roles/secretmanager.secretAccessor"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${INGESTION_SA}" \
```

```
  --role="roles/pubsub.publisher"

# BKT service - Firestore read/write only
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${BKT_SA}" \
  --role="roles/datastore.user"

# GenUI service - Secret Manager for API keys
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${GENUI_SA}" \
  --role="roles/secretmanager.secretAccessor"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${GENUI_SA}" \
  --role="roles/datastore.viewer"

echo "✅ Service accounts created and roles assigned"
```

3.5 Create Cloud Storage Buckets

```
#!/bin/bash
# scripts/gcp/04_create_storage.sh

set -e
PROJECT_ID=$(gcloud config get-value project)
REGION="us-central1"

# Lesson uploads bucket (teacher PPTs/PDFs)
gsutil mb -p $PROJECT_ID -c STANDARD -l $REGION gs://${PROJECT_ID}-lesson-uploads
gsutil cors set scripts/gcp/cors.json gs://${PROJECT_ID}-lesson-uploads

# Processed assets bucket (extracted images, thumbnails)
gsutil mb -p $PROJECT_ID -c STANDARD -l $REGION gs://${PROJECT_ID}-lesson-assets

# Set lifecycle policies (auto-delete uploads after 30 days)
cat > /tmp/lifecycle.json << 'EOF'
{
  "rule": [{
    "action": {"type": "Delete"},
    "condition": {"age": 30}
  }]
}
EOF
gsutil lifecycle set /tmp/lifecycle.json gs://${PROJECT_ID}-lesson-uploads

echo "✅ GCS buckets created"
```

```
// scripts/gcp/cors.json
[
  {
    "origin": ["*"],
    "method": ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    "responseHeader": ["Content-Type", "Authorization"],
    "maxAgeSeconds": 3600
  }
]
```

3.6 Create Pub/Sub Topics & Subscriptions

```bash
#!/bin/bash
# scripts/gcp/05_create_pubsub.sh

set -e
PROJECT_ID=$(gcloud config get-value project)
REGION="us-central1"

# Topic: lesson ingestion requests
gcloud pubsub topics create lesson-ingestion-requests --project=$PROJECT_ID

# Topic: ingestion completion notifications
gcloud pubsub topics create lesson-ingestion-complete --project=$PROJECT_ID

# Topic: BKT update events (for analytics aggregation)
gcloud pubsub topics create bkt-state-updates --project=$PROJECT_ID

# Subscription for ingestion worker (push to Cloud Run Job trigger)
# NOTE: Update INGESTION_JOB_URL after deploying ingestion service
INGESTION_JOB_URL="https://ingestion-trigger-${PROJECT_ID}-uc.a.run.app/trigger"
INGESTION_SA="eduforge-ingestion@${PROJECT_ID}.iam.gserviceaccount.com"

gcloud pubsub subscriptions create lesson-ingestion-sub \
  --topic=lesson-ingestion-requests \
  --push-endpoint=$INGESTION_JOB_URL \
  --push-auth-service-account=$INGESTION_SA \
  --ack-deadline=600 \
  --project=$PROJECT_ID

# Subscription for completion notifications (push to API service)
gcloud pubsub subscriptions create lesson-complete-sub \
  --topic=lesson-ingestion-complete \
  --push-endpoint="https://api-${PROJECT_ID}-uc.a.run.app/internal/lesson-complete" \
  --push-auth-service-account="eduforge-api@${PROJECT_ID}.iam.gserviceaccount.com" \
  --ack-deadline=60 \
  --project=$PROJECT_ID

echo "✅ Pub/Sub topics and subscriptions created"
```

3.7 Create Cloud Document AI Processor

```bash
#!/bin/bash
# scripts/gcp/06_create_document_ai.sh

set -e
PROJECT_ID=$(gcloud config get-value project)
LOCATION="us" # Document AI is regional: us or eu

# Create Layout Parser processor (supports PPTX natively in GA)
curl -X POST \
  -H "Authorization: Bearer $(gcloud auth print-access-token)" \
  -H "Content-Type: application/json" \
  -d '{
    "type": "LAYOUT_PARSER_PROCESSOR",
    "displayName": "EduForge Layout Parser"
  }' \
  "https://${LOCATION}-documentai.googleapis.com/v1/projects/${PROJECT_ID}/locations/${LOCATION}/processors"

echo "✅ Document AI processor created"
echo "⚠️  IMPORTANT: Copy the processor ID from the response above"
echo "   Add it to Secret Manager: DOCUMENT_AI_PROCESSOR_ID"
```

3.8 Create Secret Manager Secrets

```bash
#!/bin/bash
# scripts/gcp/07_create_secrets.sh
# Run this and fill in your actual API keys

set -e
PROJECT_ID=$(gcloud config get-value project)

create_secret() {
  local name=$1
  local value=$2
  echo -n "$value" | gcloud secrets create $name \
    --data-file=- \
    --project=$PROJECT_ID 2>/dev/null || \
  echo -n "$value" | gcloud secrets versions add $name \
    --data-file=- \
    --project=$PROJECT_ID
}

# Replace with actual values
create_secret "CLAUDE_API_KEY" "YOUR_ANTHROPIC_API_KEY_HERE"
create_secret "GEMINI_API_KEY" "YOUR_GOOGLE_AI_API_KEY_HERE"
create_secret "DOCUMENT_AI_PROCESSOR_ID" "YOUR_PROCESSOR_ID_HERE"
create_secret "DOCUMENT_AI_LOCATION" "us"
create_secret "FIREBASE_SERVICE_ACCOUNT" "$(cat firebase-service-account.json)"
create_secret "NEXTAUTH_SECRET" "$(openssl rand -base64 32)"

echo "✅ Secrets created in Secret Manager"
echo "   Grant service accounts access via 03_create_service_accounts.sh (already done)"
```

3.9 Initialize Firestore

```bash
#!/bin/bash
# scripts/gcp/08_init_firestore.sh

set -e
PROJECT_ID=$(gcloud config get-value project)

# Create Firestore database (Native mode)
gcloud firestore databases create \
  --location=us-central \
  --type=firestore-native \
  --project=$PROJECT_ID

echo "✅ Firestore database initialized in Native mode"
echo "   Deploy security rules after frontend setup:"
echo "   firebase deploy --only firestore:rules"
```

3.10 Create Cloud Tasks Queue

```bash
#!/bin/bash
# scripts/gcp/09_create_tasks.sh

set -e
PROJECT_ID=$(gcloud config get-value project)
REGION="us-central1"

# Queue for ingestion retries
gcloud tasks queues create ingestion-retry-queue \
  --location=$REGION \
  --max-attempts=3 \
  --min-backoff=30s \
  --max-backoff=300s \
  --project=$PROJECT_ID

echo "✅ Cloud Tasks queue created"
```

4. Production Directory Structure

```
eduforge/
├── README.md
├── docker-compose.yml                  # Local development
├── .env.example                        # Environment variable template
├── .env.local                          # Local dev secrets (gitignored)
├── firebase.json                       # Firebase config
├── firestore.rules                     # Firestore security rules
├── firestore.indexes.json              # Composite indexes
│
├── scripts/
│   ├── gcp/
│   │   ├── 01_enable_apis.sh
│   │   ├── 02_create_registry.sh
│   │   ├── 03_create_service_accounts.sh
│   │   ├── 04_create_storage.sh
│   │   ├── 05_create_pubsub.sh
│   │   ├── 06_create_document_ai.sh
│   │   ├── 07_create_secrets.sh
│   │   ├── 08_init_firestore.sh
│   │   ├── 09_create_tasks.sh
│   │   ├── cors.json
│   │   └── deploy_all.sh               # Master deploy script
│   ├── seed/
│   │   ├── seed_demo_lesson.py         # Creates demo data for competition
│   │   └── seed_students.py
│   └── test/
│       └── run_all_tests.sh
│
├── packages/
│   ├── shared/                         # Shared types, utils, constants
│   │   ├── package.json
│   │   ├── src/
│   │   │   ├── types/
│   │   │   │   ├── lesson.ts
│   │   │   │   ├── bkt.ts
│   │   │   │   ├── student.ts
│   │   │   │   └── genui.ts
│   │   │   ├── constants/
│   │   │   │   ├── scaffold-levels.ts
│   │   │   │   └── bkt-thresholds.ts
│   │   │   └── utils/
│   │   │       └── bkt-helpers.ts
│   │   └── tsconfig.json
│
├── apps/
│   ├── web/                            # Next.js 15 frontend
│   │   ├── package.json
│   │   ├── next.config.ts
│   │   ├── tailwind.config.ts
│   │   ├── tsconfig.json
│   │   ├── public/
│   │   ├── src/
│   │   │   ├── app/                    # Next.js App Router
│   │   │   │   ├── (auth)/
│   │   │   │   │   ├── login/
│   │   │   │   │   │   └── page.tsx
│   │   │   │   │   └── register/
│   │   │   │   │       └── page.tsx
│   │   │   │   ├── (teacher)/
│   │   │   │   │   ├── layout.tsx
│   │   │   │   │   ├── dashboard/
│   │   │   │   │   │   └── page.tsx
│   │   │   │   │   ├── lessons/
│   │   │   │   │   │   ├── page.tsx
│   │   │   │   │   │   ├── new/
│   │   │   │   │   │   │   └── page.tsx
│   │   │   │   │   │   └── [id]/
│   │   │   │   │   │       ├── page.tsx
│   │   │   │   │   │       └── analytics/
│   │   │   │   │   │           └── page.tsx
│   │   │   │   │   └── students/
│   │   │   │   │       └── page.tsx
│   │   │   │   ├── (student)/
│   │   │   │   │   ├── layout.tsx
│   │   │   │   │   ├── dashboard/
```

```
│   │   │   │   │   │   └── page.tsx
│   │   │   │   │   └── learn/
│   │   │   │   │       └── [lessonId]/
│   │   │   │   │           ├── page.tsx
│   │   │   │   │           └── [subtopicId]/
│   │   │   │   │               └── page.tsx
│   │   │   │   ├── api/
│   │   │   │   │   └── genui/
│   │   │   │   │       └── route.ts   # GenUI streaming endpoint
│   │   │   │   ├── layout.tsx
│   │   │   │   └── page.tsx
│   │   │   ├── components/
│   │   │   │   ├── ui/                # shadcn/ui components
│   │   │   │   ├── teacher/
│   │   │   │   │   ├── LessonUploader.tsx
│   │   │   │   │   ├── IngestionStatus.tsx
│   │   │   │   │   ├── ConceptEditor.tsx
│   │   │   │   │   ├── ClassHeatmap.tsx
│   │   │   │   │   └── StudentProgressTable.tsx
│   │   │   │   ├── student/
│   │   │   │   │   ├── TopicIndex.tsx
│   │   │   │   │   ├── SubtopicCard.tsx
│   │   │   │   │   ├── MasteryHUD.tsx
│   │   │   │   │   ├── AdaptiveMCQ.tsx
│   │   │   │   │   ├── AITutorChat.tsx
│   │   │   │   │   └── GenUIPanel.tsx
│   │   │   │   ├── genui/
│   │   │   │   │   ├── components/    # Registered OpenUI components
│   │   │   │   │   │   ├── StepByStep.tsx
│   │   │   │   │   │   ├── ConceptDiagram.tsx
│   │   │   │   │   │   ├── FormulaCard.tsx
│   │   │   │   │   │   ├── HintCard.tsx
│   │   │   │   │   │   ├── AnalogyCaed.tsx
│   │   │   │   │   │   ├── ProofWalkthrough.tsx
│   │   │   │   │   │   ├── PracticeExercise.tsx
│   │   │   │   │   │   └── ExpertSummary.tsx
│   │   │   │   │   ├── library.ts     # OpenUI component library definition
│   │   │   │   │   └── renderer.tsx   # OpenUI renderer wrapper
│   │   │   │   └── shared/
│   │   │   │       ├── LoadingStates.tsx
│   │   │   │       ├── ErrorBoundary.tsx
│   │   │   │       └── ProgressRing.tsx
│   │   │   ├── hooks/
│   │   │   │   ├── useBKTState.ts      # Firestore real-time BKT listener
│   │   │   │   ├── useLesson.ts
│   │   │   │   ├── useGenUI.ts         # GenUI streaming hook
│   │   │   │   └── useIngestionStatus.ts
│   │   │   ├── lib/
│   │   │   │   ├── firebase.ts         # Firebase client init
│   │   │   │   ├── firestore.ts        # Typed Firestore helpers
│   │   │   │   ├── auth.ts             # Firebase Auth helpers
│   │   │   │   └── api.ts              # API client
│   │   │   └── stores/
│   │   │       ├── bktStore.ts         # Zustand BKT state store
│   │   │       └── sessionStore.ts
│   │   └── Dockerfile
│   │
│   ├── api/                            # FastAPI API Gateway (Cloud Run)
│   │   ├── Dockerfile
│   │   ├── requirements.txt
│   │   ├── main.py
│   │   ├── src/
│   │   │   ├── routers/
│   │   │   │   ├── lessons.py
│   │   │   │   ├── students.py
│   │   │   │   ├── bkt.py
│   │   │   │   ├── analytics.py
│   │   │   │   └── internal.py        # Internal Pub/Sub callbacks
│   │   │   ├── services/
│   │   │   │   ├── firestore_service.py
│   │   │   │   ├── storage_service.py
│   │   │   │   └── pubsub_service.py
│   │   │   ├── middleware/
│   │   │   │   ├── auth.py             # Firebase JWT verification
│   │   │   │   └── rate_limit.py
│   │   │   └── models/
│   │   │       ├── lesson.py
│   │   │       ├── bkt.py
│   │   │       └── student.py
│   │   └── tests/
```

```
│   │       ├── test_bkt_router.py
│   │       ├── test_lessons.py
│   │       └── conftest.py
│   │
│   ├── bkt-service/                  # BKT Engine (Cloud Run)
│   │   ├── Dockerfile
│   │   ├── requirements.txt
│   │   ├── main.py
│   │   ├── src/
│   │   │   ├── bkt/
│   │   │   │   ├── engine.py           # pyBKT wrapper
│   │   │   │   ├── scaffold_resolver.py # P(mastery) → scaffold level
│   │   │   │   ├── component_gating.py # Scaffold level → allowed components
│   │   │   │   └── misconception_detector.py
│   │   │   ├── routers/
│   │   │   │   ├── update.py           # POST /update — after MCQ answer
│   │   │   │   ├── state.py            # GET /state — current BKT state
│   │   │   │   └── scaffold.py         # GET /scaffold — current scaffold level
│   │   │   └── services/
│   │   │       └── firestore_service.py
│   │   └── tests/
│   │       ├── test_bkt_engine.py
│   │       ├── test_scaffold_resolver.py
│   │       └── fixtures/
│   │           └── sample_responses.json
│   │
│   ├── ingestion/                    # PPT Processing (Cloud Run Job)
│   │   ├── Dockerfile
│   │   ├── requirements.txt
│   │   ├── main.py                   # Entry point (triggered by Pub/Sub)
│   │   ├── src/
│   │   │   ├── extractors/
│   │   │   │   ├── pptx_extractor.py   # python-pptx slide extraction
│   │   │   │   ├── document_ai.py      # Cloud Document AI layout parser
│   │   │   │   └── image_processor.py  # Image extraction + GCS upload
│   │   │   ├── generators/
│   │   │   │   ├── topic_hierarchy.py  # Gemini: extract topic tree
│   │   │   │   ├── mcq_generator.py    # Gemini: generate MCQ bank
│   │   │   │   ├── concept_graph.py    # Build concept dependency graph
│   │   │   │   └── bkt_params.py       # Seed initial BKT parameters
│   │   │   └── services/
│   │   │       ├── firestore_service.py
│   │   │       ├── storage_service.py
│   │   │       └── pubsub_service.py
│   │   └── tests/
│   │       ├── test_pptx_extractor.py
│   │       ├── test_mcq_generator.py
│   │       └── fixtures/
│   │           └── sample.pptx
│   │
│   └── genui-service/                # GenUI Streaming (Cloud Run)
│       ├── Dockerfile
│       ├── requirements.txt          # OR package.json if Node.js
│       ├── main.py
│       ├── src/
│       │   ├── genui/
│       │   │   ├── catalog.py          # Component catalog definitions
│       │   │   ├── prompt_builder.py   # BKT state → system prompt
│       │   │   └── streamer.py         # Claude API streaming
│       │   └── routers/
│       │       ├── visualize.py        # POST /genui/visualize
│       │       └── chat.py             # POST /genui/chat (AI tutor)
│       └── tests/
│           └── test_prompt_builder.py
```

---

5. Feature Specifications — Teacher Portal

5.1 Authentication & Role Management

Firebase Auth Implementation:

```typescript
// apps/web/src/lib/auth.ts
import {
  getAuth,
  signInWithEmailAndPassword,
  createUserWithEmailAndPassword,
  GoogleAuthProvider,
  signInWithPopup
} from 'firebase/auth';
import { doc, setDoc, getDoc } from 'firebase/firestore';
import { db } from './firebase';

export type UserRole = 'teacher' | 'student';

export interface UserProfile {
  uid: string;
  email: string;
  displayName: string;
  role: UserRole;
  institution?: string;
  createdAt: Date;
}

export async function registerUser(
  email: string,
  password: string,
  displayName: string,
  role: UserRole,
  institution?: string
): Promise<UserProfile> {
  const auth = getAuth();
  const { user } = await createUserWithEmailAndPassword(auth, email, password);

  const profile: UserProfile = {
    uid: user.uid,
    email: user.email!,
    displayName,
    role,
    institution,
    createdAt: new Date(),
  };

  // Store profile in Firestore with role
  await setDoc(doc(db, 'users', user.uid), profile);
  return profile;
}

export async function getUserProfile(uid: string): Promise<UserProfile | null> {
  const snap = await getDoc(doc(db, 'users', uid));
  return snap.exists() ? snap.data() as UserProfile : null;
}
```

5.2 Lesson Creation & PPT Upload

Upload Flow with Signed URLs:

```
// apps/web/src/components/teacher/LessonUploader.tsx
'use client';

import { useState, useCallback } from 'react';
import { useDropzone } from 'react-dropzone';
import { useRouter } from 'next/navigation';

interface UploadState {
  status: 'idle' | 'uploading' | 'processing' | 'complete' | 'error';
  progress: number;
  lessonId?: string;
  message?: string;
}

export function LessonUploader() {
  const [uploadState, setUploadState] = useState<UploadState>({
    status: 'idle',
    progress: 0
  });
  const [lessonTitle, setLessonTitle] = useState('');
  const [subject, setSubject] = useState('');
  const router = useRouter();

  const onDrop = useCallback(async (acceptedFiles: File[]) => {
    const file = acceptedFiles[0];
    if (!file) return;

    setUploadState({ status: 'uploading', progress: 0 });

    try {
      // Step 1: Get signed upload URL from API
      const { uploadUrl, lessonId, gcsPath } = await fetch('/api/lessons/upload-url', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          filename: file.name,
          contentType: file.type,
          lessonTitle,
          subject
        }),
      }).then(r => r.json());

      // Step 2: Upload directly to GCS using signed URL (bypasses API size limits)
      await uploadToGCS(uploadUrl, file, (progress) => {
        setUploadState(s => ({ ...s, progress }));
      });

      // Step 3: Notify API to start processing
      await fetch('/api/lessons/start-ingestion', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ lessonId, gcsPath }),
      });

      setUploadState({
        status: 'processing',
        progress: 100,
        lessonId,
        message: 'Processing your lesson... This may take up to 60 minutes.'
      });

      // Step 4: Navigate to ingestion status page
      router.push(`/lessons/${lessonId}?status=processing`);

    } catch (error) {
      setUploadState({
        status: 'error',
        progress: 0,
        message: 'Upload failed. Please try again.'
      });
    }
  }, [lessonTitle, subject]);

  const { getRootProps, getInputProps, isDragActive } = useDropzone({
    onDrop,
    accept: {
      'application/vnd.openxmlformats-officedocument.presentationml.presentation': ['.pptx'],
      'application/pdf': ['.pdf'],
    },
```

```
    maxSize: 100 * 1024 * 1024, // 100MB
    multiple: false,
  });

  return (
    <div className="space-y-4">
      <div className="grid grid-cols-2 gap-4">
        <input
          placeholder="Lesson Title (e.g., Newton's Laws of Motion)"
          value={lessonTitle}
          onChange={e => setLessonTitle(e.target.value)}
          className="input"
        />
        <input
          placeholder="Subject (e.g., Physics)"
          value={subject}
          onChange={e => setSubject(e.target.value)}
          className="input"
        />
      </div>

      <div
        {...getRootProps()}
        className={`border-2 border-dashed rounded-xl p-12 text-center cursor-pointer transition-colors
          ${isDragActive ? 'border-blue-500 bg-blue-50' : 'border-gray-300 hover:border-gray-400'}`}
      >
        <input {...getInputProps()} />
        {uploadState.status === 'idle' && (
          <div>
            <p className="text-lg font-medium">Drop your PPTX or PDF here</p>
            <p className="text-gray-500 text-sm mt-1">Supports .pptx and .pdf up to 100MB</p>
          </div>
        )}
        {uploadState.status === 'uploading' && (
          <div>
            <p>Uploading... {uploadState.progress}%</p>
            <div className="mt-2 bg-gray-200 rounded-full h-2">
              <div
                className="bg-blue-600 h-2 rounded-full transition-all"
                style={{ width: `${uploadState.progress}%` }}
              />
            </div>
          </div>
        )}
        {uploadState.status === 'processing' && (
          <div>
            <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-blue-600 mx-auto" />
            <p className="mt-2">{uploadState.message}</p>
          </div>
        )}
      </div>
    </div>
  );
}

async function uploadToGCS(
  signedUrl: string,
  file: File,
  onProgress: (p: number) => void
): Promise<void> {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();
    xhr.upload.onprogress = (e) => {
      if (e.lengthComputable) onProgress(Math.round((e.loaded / e.total) * 100));
    };
    xhr.onload = () => xhr.status === 200 ? resolve() : reject(new Error(`Upload failed: ${xhr.status}`));
    xhr.onerror = () => reject(new Error('Network error'));
    xhr.open('PUT', signedUrl);
    xhr.setRequestHeader('Content-Type', file.type);
    xhr.send(file);
  });
}
```

5.3 Ingestion Status — Real-time Updates via Firestore

```
// apps/web/src/hooks/useIngestionStatus.ts
import { useEffect, useState } from 'react';
import { doc, onSnapshot } from 'firebase/firestore';
import { db } from '@/lib/firebase';

export type IngestionStep =
  | 'queued'
  | 'extracting'
  | 'parsing_layout'
  | 'generating_topics'
  | 'generating_mcqs'
  | 'building_graph'
  | 'complete'
  | 'failed';

export interface IngestionStatus {
  step: IngestionStep;
  progress: number; // 0-100
  message: string;
  subtopicsFound?: number;
  mcqsGenerated?: number;
  conceptsFound?: number;
  error?: string;
  completedAt?: Date;
}

export function useIngestionStatus(lessonId: string) {
  const [status, setStatus] = useState<IngestionStatus | null>(null);

  useEffect(() => {
    // Firestore real-time listener — updates as ingestion progresses
    const unsubscribe = onSnapshot(
      doc(db, 'lessons', lessonId),
      (snap) => {
        if (snap.exists()) {
          const data = snap.data();
          setStatus(data.ingestion as IngestionStatus);
        }
      }
    );
    return unsubscribe;
  }, [lessonId]);

  return status;
}
```

5.4 Concept Graph Editor

After ingestion completes, the teacher can view and edit the auto-generated concept dependency graph before publishing.

```
// apps/web/src/components/teacher/ConceptEditor.tsx
// Uses React Flow for the visual graph editor

import ReactFlow, {
  Node, Edge,
  Background, Controls, MiniMap,
  addEdge, useNodesState, useEdgesState
} from 'reactflow';

interface ConceptNode {
  id: string;
  label: string;
  description: string;
  subtopicId: string;
  prerequisites: string[]; // IDs of prerequisite concept nodes
  difficulty: 'foundational' | 'intermediate' | 'advanced';
}

// Teacher can:
// 1. Rename auto-generated concept nodes
// 2. Add/remove prerequisite edges (controls BKT unlock order)
// 3. Merge duplicate concepts
// 4. Adjust MCQ difficulty ratings
// 5. Preview the MCQ bank for each concept (edit/regenerate individual MCQs)
```

---

6. Feature Specifications — Async Processing Pipeline

6.1 Architecture: Cloud Run Job + Pub/Sub

The ingestion pipeline uses **Cloud Run Jobs** (not Services) because:

- No HTTP timeout limit (Services max out at 60 minutes)
- Designed for batch processing
- Triggered via Pub/Sub push → a lightweight Cloud Run **Service** receives the Pub/Sub message and triggers the Job

```
Teacher uploads PPT
        │
        ▼
API: Create lesson doc (status: queued) → Upload to GCS → Publish to Pub/Sub
        │
        ▼ (Pub/Sub push subscription)
ingestion-trigger Service (lightweight FastAPI)
        │
        ▼ (Trigger Cloud Run Job)
ingestion-job (Cloud Run Job, timeout: 3600s)
        │
        ├── Extract PPTX slides (python-pptx)
        ├── Cloud Document AI Layout Parser
        ├── Gemini API: Generate topic hierarchy
        ├── Gemini API: Generate MCQ bank (parallel per subtopic)
        ├── Build concept dependency graph
        ├── Seed BKT parameters
        └── Write to Firestore → Pub/Sub completion notification
```

6.2 PPTX Extraction

```python
# apps/ingestion/src/extractors/pptx_extractor.py
from pptx import Presentation
from pptx.util import Inches
from google.cloud import storage
import io
import base64
from dataclasses import dataclass
from typing import Optional

@dataclass
class SlideContent:
    slide_number: int
    title: Optional[str]
    body_text: list[str]
    notes: Optional[str]
    image_paths: list[str]  # GCS paths after upload
    raw_text: str  # All text combined

class PPTXExtractor:
    def __init__(self, project_id: str, assets_bucket: str):
        self.storage_client = storage.Client(project=project_id)
        self.bucket = self.storage_client.bucket(assets_bucket)

    def extract(self, pptx_bytes: bytes, lesson_id: str) -> list[SlideContent]:
        prs = Presentation(io.BytesIO(pptx_bytes))
        slides = []

        for i, slide in enumerate(prs.slides):
            title = None
            body_text = []

            for shape in slide.shapes:
                if hasattr(shape, 'text'):
                    if shape.shape_type == 13:  # Title placeholder
                        title = shape.text.strip()
                    elif shape.text.strip():
                        body_text.append(shape.text.strip())

            # Extract speaker notes
            notes = None
            if slide.has_notes_slide:
                notes_frame = slide.notes_slide.notes_text_frame
                if notes_frame:
                    notes = notes_frame.text.strip()

            # Extract images and upload to GCS
            image_paths = []
            for shape in slide.shapes:
                if shape.shape_type == 13 or not hasattr(shape, 'image'):
                    continue
                try:
                    image = shape.image
                    ext = image.ext  # 'png', 'jpg', etc.
                    img_bytes = image.blob
                    gcs_path = f"lessons/{lesson_id}/slides/slide_{i+1}_{shape.shape_id}.{ext}"

                    blob = self.bucket.blob(gcs_path)
                    blob.upload_from_string(img_bytes, content_type=f"image/{ext}")
                    image_paths.append(f"gs://{self.bucket.name}/{gcs_path}")
                except Exception:
                    pass  # Skip corrupted images

            slides.append(SlideContent(
                slide_number=i + 1,
                title=title,
                body_text=body_text,
                notes=notes,
                image_paths=image_paths,
                raw_text=f"{title or ''}\n{chr(10).join(body_text)}\n{notes or ''}"
            ))

        return slides
```

6.3 Cloud Document AI Integration

```python
# apps/ingestion/src/extractors/document_ai.py
from google.cloud import documentai
from google.api_core.client_options import ClientOptions
import os

class DocumentAIParser:
    """
    Uses Cloud Document AI Layout Parser (GA 2025)
    Supports PPTX natively — extracts paragraphs, headings, tables, lists
    Ref: https://cloud.google.com/document-ai/docs/release-notes
    Model: pretrained-layout-parser-v1.5-2025-08-25
    """

    def __init__(self):
        self.project_id = os.environ['GOOGLE_CLOUD_PROJECT']
        self.location = os.environ.get('DOCUMENT_AI_LOCATION', 'us')
        self.processor_id = os.environ['DOCUMENT_AI_PROCESSOR_ID']

        opts = ClientOptions(
            api_endpoint=f"{self.location}-documentai.googleapis.com"
        )
        self.client = documentai.DocumentProcessorServiceClient(
            client_options=opts
        )
        self.processor_name = (
            f"projects/{self.project_id}/locations/{self.location}"
            f"/processors/{self.processor_id}"
        )

    def parse_document(self, file_bytes: bytes, mime_type: str) -> dict:
        """
        Parses a PPTX or PDF using Document AI Layout Parser.
        Returns structured content with paragraphs, headings, tables.
        """
        raw_document = documentai.RawDocument(
            content=file_bytes,
            mime_type=mime_type,  # 'application/vnd.openxmlformats-officedocument.presentationml.presentation'
        )

        request = documentai.ProcessRequest(
            name=self.processor_name,
            raw_document=raw_document,
        )

        result = self.client.process_document(request=request)
        document = result.document

        # Extract structured content
        structured = {
            'full_text': document.text,
            'pages': [],
        }

        for page in document.pages:
            page_content = {
                'page_number': page.page_number,
                'paragraphs': [],
                'tables': [],
                'headings': [],
            }

            for block in page.blocks:
                text = self._extract_text(document, block.layout)
                if text.strip():
                    page_content['paragraphs'].append(text.strip())

            for table in page.tables:
                table_data = []
                for row in table.header_rows:
                    table_data.append([
                        self._extract_text(document, cell.layout)
                        for cell in row.cells
                    ])
                for row in table.body_rows:
                    table_data.append([
                        self._extract_text(document, cell.layout)
```

```
                        for cell in row.cells
                    ])
                page_content['tables'].append(table_data)

            structured['pages'].append(page_content)

        return structured

    def _extract_text(self, document, layout) -> str:
        """Extract text from a layout segment."""
        text = ''
        for segment in layout.text_anchor.text_segments:
            start = int(segment.start_index) if segment.start_index else 0
            end = int(segment.end_index)
            text += document.text[start:end]
        return text
```

6.4 Gemini-Powered Topic Hierarchy + MCQ Generation

```python
# apps/ingestion/src/generators/topic_hierarchy.py
import google.generativeai as genai
import json
import os
from typing import TypedDict

class Subtopic(TypedDict):
    id: str
    title: str
    description: str
    slide_numbers: list[int]
    key_concepts: list[str]
    learning_objectives: list[str]
    prerequisite_subtopic_ids: list[str]
    difficulty: str  # 'foundational' | 'intermediate' | 'advanced'

class TopicHierarchyGenerator:
    def __init__(self):
        genai.configure(api_key=os.environ['GEMINI_API_KEY'])
        self.model = genai.GenerativeModel('gemini-2.0-flash')

    def generate(self, slide_contents: list[dict], lesson_title: str) -> list[Subtopic]:
        """
        Uses Gemini to identify the topic hierarchy from extracted slide content.
        Returns a structured list of subtopics with prerequisites.
        """
        slides_text = "\n\n".join([
            f"=== SLIDE {s['slide_number']}: {s.get('title', 'Untitled')} ===\n{s['raw_text']}"
            for s in slide_contents
        ])

        prompt = f"""You are an expert educator analyzing a lesson presentation titled "{lesson_title}".

Here is the content from all slides:

{slides_text}

Your task: Extract a structured topic hierarchy for this lesson.

Return a JSON array of subtopics. Each subtopic should represent a cohesive learning unit (usually 2-5 slides worth of content).

For each subtopic, identify:
1. A clear title (concise, specific)
2. Description (1-2 sentences)
3. Which slide numbers it covers
4. 3-5 key concepts a student must understand
5. 2-3 specific learning objectives (what the student will be able to DO)
6. Prerequisites: IDs of subtopics that MUST be mastered before this one (use sequential IDs like "subtopic_1")
7. Difficulty: "foundational", "intermediate", or "advanced"

Return ONLY valid JSON, no other text:
{{
  "subtopics": [
    {{
      "id": "subtopic_1",
      "title": "string",
      "description": "string",
      "slide_numbers": [1, 2, 3],
      "key_concepts": ["concept1", "concept2"],
      "learning_objectives": ["Students will be able to..."],
      "prerequisite_subtopic_ids": [],
      "difficulty": "foundational"
    }}
  ]
}}"""

        response = self.model.generate_content(
            prompt,
            generation_config=genai.GenerationConfig(
                response_mime_type="application/json",
                temperature=0.3,
            )
        )

        data = json.loads(response.text)
        return data['subtopics']
```

```python
# apps/ingestion/src/generators/mcq_generator.py
class MCQGenerator:
    """
    Generates a bank of MCQs per subtopic at 3 difficulty tiers.
    Tier 1 (P_mastery < 0.3): Recall/recognition questions
    Tier 2 (P_mastery 0.3-0.6): Understanding/application
    Tier 3 (P_mastery > 0.6): Analysis/synthesis
    """

    QUESTIONS_PER_TIER = 5  # 15 total per subtopic

    def __init__(self):
        genai.configure(api_key=os.environ['GEMINI_API_KEY'])
        self.model = genai.GenerativeModel('gemini-2.0-flash')

    def generate_for_subtopic(
        self,
        subtopic: dict,
        slide_content: str
    ) -> list[dict]:
        """Generate 15 MCQs (5 per tier) for a subtopic."""

        prompt = f"""You are an expert educator creating MCQs for the subtopic: "{subtopic['title']}"

Content:
{slide_content}

Key concepts to test: {', '.join(subtopic['key_concepts'])}
Learning objectives: {', '.join(subtopic['learning_objectives'])}

Generate EXACTLY 15 multiple-choice questions:
- 5 TIER_1 questions: recall/recognition (simple, direct)
- 5 TIER_2 questions: understanding/application (moderate, requires comprehension)
- 5 TIER_3 questions: analysis/synthesis (challenging, requires deep understanding)

For each question:
- 4 answer options (A, B, C, D)
- Exactly 1 correct answer
- Clear explanation for why correct answer is right
- Brief explanation for why each wrong answer is incorrect (for misconception feedback)
- The specific concept being tested (from the key_concepts list)

Return ONLY valid JSON:
{{
  "mcqs": [
    {{
      "id": "mcq_1",
      "tier": 1,
      "question": "string",
      "options": {{"A": "string", "B": "string", "C": "string", "D": "string"}},
      "correct_answer": "A",
      "concept": "string",
      "explanation": "The correct answer is A because...",
      "misconceptions": {{
        "B": "B is wrong because...",
        "C": "C is wrong because...",
        "D": "D is wrong because..."
      }}
    }}
  ]
}}"""

        response = self.model.generate_content(
            prompt,
            generation_config=genai.GenerationConfig(
                response_mime_type="application/json",
                temperature=0.4,
            )
        )

        data = json.loads(response.text)
        return data['mcqs']
```

6.5 Main Ingestion Job Entry Point

```python
# apps/ingestion/main.py
import os
import json
import base64
import asyncio
from fastapi import FastAPI, Request, HTTPException
from google.cloud import storage, firestore
from src.extractors.pptx_extractor import PPTXExtractor
from src.extractors.document_ai import DocumentAIParser
from src.generators.topic_hierarchy import TopicHierarchyGenerator
from src.generators.mcq_generator import MCQGenerator
from src.generators.bkt_params import BKTParamSeeder
from src.services.firestore_service import FirestoreService
from src.services.pubsub_service import PubSubService
import logging

app = FastAPI()
logger = logging.getLogger(__name__)

PROJECT_ID = os.environ['GOOGLE_CLOUD_PROJECT']

@app.post("/trigger")
async def trigger_ingestion(request: Request):
    """
    Receives Pub/Sub push message and starts ingestion.
    Pub/Sub base64-encodes the message body.
    """
    body = await request.json()

    try:
        # Decode Pub/Sub message
        pubsub_message = body.get('message', {})
        message_data = base64.b64decode(pubsub_message.get('data', '')).decode('utf-8')
        payload = json.loads(message_data)

        lesson_id = payload['lesson_id']
        gcs_path = payload['gcs_path']  # gs://bucket/lessons/xxx/file.pptx

        logger.info(f"Starting ingestion for lesson {lesson_id}")

        # Run ingestion pipeline
        await run_ingestion(lesson_id, gcs_path)

        return {"status": "ok"}

    except Exception as e:
        logger.error(f"Ingestion failed: {e}")
        raise HTTPException(status_code=500, detail=str(e))


async def run_ingestion(lesson_id: str, gcs_path: str):
    """Full ingestion pipeline — runs end to end."""

    fs = FirestoreService(PROJECT_ID)
    ps = PubSubService(PROJECT_ID)
    storage_client = storage.Client(project=PROJECT_ID)

    async def update_status(step: str, progress: int, message: str, **kwargs):
        await fs.update_lesson_ingestion(lesson_id, {
            'step': step,
            'progress': progress,
            'message': message,
            **kwargs
        })

    try:
        # Step 1: Download from GCS
        await update_status('extracting', 5, 'Downloading presentation...')
        bucket_name, blob_name = gcs_path.replace('gs://', '').split('/', 1)
        blob = storage_client.bucket(bucket_name).blob(blob_name)
        file_bytes = blob.download_as_bytes()

        # Step 2: Extract with python-pptx
        await update_status('extracting', 15, 'Extracting slide content...')
        extractor = PPTXExtractor(PROJECT_ID, f"{PROJECT_ID}-lesson-assets")
        slides = extractor.extract(file_bytes, lesson_id)
```

```
        # Step 3: Cloud Document AI layout parsing
        await update_status('parsing_layout', 30, 'Parsing document structure...')
        dai_parser = DocumentAIParser()
        structured_content = dai_parser.parse_document(
            file_bytes,
            'application/vnd.openxmlformats-officedocument.presentationml.presentation'
        )

        # Step 4: Generate topic hierarchy
        await update_status('generating_topics', 45, 'Identifying topics and subtopics...')
        topic_gen = TopicHierarchyGenerator()
        slide_dicts = [vars(s) for s in slides]
        subtopics = topic_gen.generate(slide_dicts, lesson_id)

        # Step 5: Store subtopics in Firestore
        for subtopic in subtopics:
            await fs.create_subtopic(lesson_id, subtopic)

        # Step 6: Generate MCQs (parallel per subtopic)
        await update_status('generating_mcqs', 60, f'Generating questions for {len(subtopics)} subtopics...')
        mcq_gen = MCQGenerator()

        async def generate_mcqs_for_subtopic(subtopic):
            # Find relevant slide content for this subtopic
            relevant_slides = [
                s for s in slides
                if s.slide_number in subtopic.get('slide_numbers', [])
            ]
            content = "\n\n".join(s.raw_text for s in relevant_slides)
            mcqs = mcq_gen.generate_for_subtopic(subtopic, content)
            await fs.create_mcqs(lesson_id, subtopic['id'], mcqs)

        await asyncio.gather(*[generate_mcqs_for_subtopic(st) for st in subtopics])

        # Step 7: Seed BKT parameters
        await update_status('building_graph', 85, 'Building knowledge graph...')
        bkt_seeder = BKTParamSeeder()
        bkt_params = bkt_seeder.seed_for_lesson(subtopics)
        await fs.store_bkt_params(lesson_id, bkt_params)

        # Step 8: Complete
        await update_status('complete', 100, 'Lesson ready!',
            subtopicsFound=len(subtopics),
            mcqsGenerated=len(subtopics) * 15,
            conceptsFound=sum(len(st['key_concepts']) for st in subtopics)
        )

        # Publish completion notification
        await ps.publish('lesson-ingestion-complete', {
            'lesson_id': lesson_id,
            'status': 'complete'
        })

        logger.info(f"✅ Ingestion complete for lesson {lesson_id}")

    except Exception as e:
        await update_status('failed', 0, str(e), error=str(e))
        raise
```

---

7. Feature Specifications — BKT Engine

7.1 BKT Theory & Implementation

BKT is a Hidden Markov Model with 4 parameters per skill:

Parameter	Symbol	Meaning	Typical Range
Initial knowledge	$P(L_0)$	Probability student already knows the concept	0.1 – 0.4
Learning rate	P(T)	Probability of learning on each practice opportunity	0.1 – 0.4
Slip rate	P(S)	Probability of wrong answer despite knowing	0.05 – 0.2
Guess rate	P(G)	Probability of right answer without knowing	0.1 – 0.3

Update equations (Bayes' rule):

```
After CORRECT answer:
P(Ln|correct) = P(Ln-1) * (1 - P(S)) / P(correct)
              where P(correct) = P(Ln-1)*(1-P(S)) + (1-P(Ln-1))*P(G)

After INCORRECT answer:
P(Ln|incorrect) = P(Ln-1) * P(S) / P(incorrect)
                where P(incorrect) = P(Ln-1)*P(S) + (1-P(Ln-1))*(1-P(G))

Then apply learning:
P(Ln+1) = P(Ln|observation) + (1 - P(Ln|observation)) * P(T)

Mastery threshold: P(L) >= 0.95 (standard in ITS literature)
```

7.2 pyBKT Engine Wrapper

```python
# apps/bkt-service/src/bkt/engine.py
import numpy as np
from dataclasses import dataclass, field
from typing import Optional
import json

@dataclass
class BKTParams:
    """BKT parameters for a single concept."""
    concept_id: str
    p_initial: float = 0.2   # P(L_0)
    p_learn: float = 0.2     # P(T) - learning rate
    p_slip: float = 0.1      # P(S) - slip rate
    p_guess: float = 0.25    # P(G) - guess rate

@dataclass
class StudentConceptState:
    """Current BKT state for one student on one concept."""
    student_id: str
    concept_id: str
    subtopic_id: str
    p_mastery: float = 0.2          # Current P(L)
    attempts: int = 0               # Total attempts
    correct_streak: int = 0         # Consecutive correct answers
    mastered: bool = False          # P(L) >= mastery_threshold
    consecutive_wrong: int = 0      # For misconception detection
    response_history: list = field(default_factory=list)


class BKTEngine:
    """
    Core BKT computation engine.
    Wraps the Bayesian update equations — no external library needed for online updates.
    pyBKT is used for batch parameter fitting from historical data.
    """

    MASTERY_THRESHOLD = 0.95

    def __init__(self, params: BKTParams):
        self.params = params

    def update(
        self,
        state: StudentConceptState,
        is_correct: bool
    ) -> StudentConceptState:
        """
        Bayesian update after student answers a question.
        Returns updated state (immutable — creates new state).
        """
        p_l = state.p_mastery
        p_t = self.params.p_learn
        p_s = self.params.p_slip
        p_g = self.params.p_guess

        # Step 1: Update P(L|observation) via Bayes
        if is_correct:
            p_correct = p_l * (1 - p_s) + (1 - p_l) * p_g
            p_l_given_obs = (p_l * (1 - p_s)) / p_correct
        else:
            p_incorrect = p_l * p_s + (1 - p_l) * (1 - p_g)
            p_l_given_obs = (p_l * p_s) / p_incorrect

        # Step 2: Apply learning opportunity
        p_l_new = p_l_given_obs + (1 - p_l_given_obs) * p_t

        # Clamp to valid probability range
        p_l_new = max(0.001, min(0.999, p_l_new))

        # Update streaks and history
        new_streak = state.correct_streak + 1 if is_correct else 0
        new_consecutive_wrong = 0 if is_correct else state.consecutive_wrong + 1

        response_history = state.response_history + [{
            'is_correct': is_correct,
            'p_mastery_before': state.p_mastery,
```

```
            'p_mastery_after': p_l_new,
        }]

        return StudentConceptState(
            student_id=state.student_id,
            concept_id=state.concept_id,
            subtopic_id=state.subtopic_id,
            p_mastery=p_l_new,
            attempts=state.attempts + 1,
            correct_streak=new_streak,
            mastered=p_l_new >= self.MASTERY_THRESHOLD,
            consecutive_wrong=new_consecutive_wrong,
            response_history=response_history[-20:],  # Keep last 20
        )

    def predict_next_correct(self, state: StudentConceptState) -> float:
        """Predict probability of correct answer on next question."""
        p_l = state.p_mastery
        return p_l * (1 - self.params.p_slip) + (1 - p_l) * self.params.p_guess
```

7.3 Scaffold Resolver

```python
# apps/bkt-service/src/bkt/scaffold_resolver.py
from dataclasses import dataclass
from typing import NamedTuple

class ScaffoldLevel(NamedTuple):
    level: int           # 0-4
    name: str
    p_mastery_range: tuple[float, float]
    description: str

SCAFFOLD_LEVELS = [
    ScaffoldLevel(0, "novice",        (0.0,  0.2),  "Complete beginner — full guided walkthrough"),
    ScaffoldLevel(1, "developing",    (0.2,  0.4),  "Some awareness — structured scaffold with hints"),
    ScaffoldLevel(2, "approaching",   (0.4,  0.6),  "Partial understanding — hints available on request"),
    ScaffoldLevel(3, "proficient",    (0.6,  0.8),  "Good understanding — minimal scaffold, practice focus"),
    ScaffoldLevel(4, "mastered",      (0.8,  1.0),  "Expert — challenge mode, edge cases, Socratic only"),
]

# Maps scaffold level → allowed OpenUI component catalog
COMPONENT_CATALOG: dict[int, list[str]] = {
    0: [
        "StepByStep",       # Numbered guided steps
        "HintCard",         # Explicit hint with full explanation
        "FormulaCard",      # Formula reference card (always visible)
        "AnalogyCard",      # Real-world analogy for abstract concepts
    ],
    1: [
        "StepByStep",
        "HintCard",
        "FormulaCard",
        "ConceptDiagram",   # Visual diagram (partial labels)
    ],
    2: [
        "ConceptDiagram",   # Full labeled diagram
        "FormulaCard",
        "HintCard",         # Available but student must request
        "PracticeExercise", # Practice with immediate feedback
    ],
    3: [
        "ConceptDiagram",
        "PracticeExercise",
        "ProofWalkthrough",  # Step-through mathematical proof
    ],
    4: [
        "ConceptDiagram",
        "ExpertSummary",    # Dense expert-level summary
        "ProofWalkthrough",
        "PracticeExercise",
    ],
}

@dataclass
class ScaffoldDecision:
    level: int
    level_name: str
    allowed_components: list[str]
    p_mastery: float
    description: str
    system_prompt_fragment: str  # Injected into LLM system prompt


class ScaffoldResolver:
    def resolve(self, p_mastery: float) -> ScaffoldDecision:
        """Maps P(mastery) to scaffold level and allowed components."""

        level = 0
        for sl in SCAFFOLD_LEVELS:
            if sl.p_mastery_range[0] <= p_mastery < sl.p_mastery_range[1]:
                level = sl.level
                level_info = sl
                break
        else:
            level = 4
            level_info = SCAFFOLD_LEVELS[4]

        components = COMPONENT_CATALOG[level]
```

```
        system_prompt_fragment = self._build_prompt_fragment(level, p_mastery, components)

        return ScaffoldDecision(
            level=level,
            level_name=level_info.name,
            allowed_components=components,
            p_mastery=p_mastery,
            description=level_info.description,
            system_prompt_fragment=system_prompt_fragment,
        )

    def _build_prompt_fragment(
        self,
        level: int,
        p_mastery: float,
        components: list[str]
    ) -> str:
        fragments = [
            f"STUDENT_MASTERY_LEVEL: {level}/4 (P(mastery)={p_mastery:.2f})",
            f"AVAILABLE_COMPONENTS: {', '.join(components)}",
        ]

        if level == 0:
            fragments.append("INSTRUCTION: Student is a complete beginner. Use StepByStep with small, numbered steps. Always show Formul
        elif level == 1:
            fragments.append("INSTRUCTION: Student is developing. Break down concepts clearly. Provide hints proactively. Show FormulaCa
        elif level == 2:
            fragments.append("INSTRUCTION: Student is approaching mastery. Show complete diagrams. Hints are available but don't push th
        elif level == 3:
            fragments.append("INSTRUCTION: Student is proficient. Minimal scaffolding. Focus on practice and application. Challenge with
        elif level == 4:
            fragments.append("INSTRUCTION: Student has mastered this. Use Socratic method — ask leading questions. Present edge cases an

        return "\n".join(fragments)
```

7.4 BKT Service API

```python
# apps/bkt-service/src/routers/update.py
from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from ..bkt.engine import BKTEngine, StudentConceptState, BKTParams
from ..bkt.scaffold_resolver import ScaffoldResolver
from ..bkt.misconception_detector import MisconceptionDetector
from ..services.firestore_service import FirestoreService

router = APIRouter()

class MCQAnswerRequest(BaseModel):
    student_id: str
    lesson_id: str
    subtopic_id: str
    concept_id: str
    mcq_id: str
    selected_answer: str  # 'A', 'B', 'C', 'D'
    is_correct: bool
    time_taken_seconds: int

class BKTUpdateResponse(BaseModel):
    p_mastery_before: float
    p_mastery_after: float
    scaffold_level: int
    allowed_components: list[str]
    mastered: bool
    misconception: dict | None  # If detected
    next_action: str  # 'continue' | 'subtopic_complete' | 'prerequisite_remediation'

@router.post("/update", response_model=BKTUpdateResponse)
async def update_bkt_state(
    request: MCQAnswerRequest,
    fs: FirestoreService = Depends()
):
    """
    Called after every MCQ answer. Updates BKT state, resolves scaffold level,
    detects misconceptions, and returns the new allowed GenUI components.
    """

    # Load current state and params from Firestore
    current_state = await fs.get_student_concept_state(
        request.student_id, request.concept_id
    )
    bkt_params = await fs.get_bkt_params(request.lesson_id, request.concept_id)

    if not current_state:
        # Initialize state for new student-concept pair
        current_state = StudentConceptState(
            student_id=request.student_id,
            concept_id=request.concept_id,
            subtopic_id=request.subtopic_id,
            p_mastery=bkt_params.p_initial,
        )

    p_mastery_before = current_state.p_mastery

    # Run BKT update
    engine = BKTEngine(bkt_params)
    new_state = engine.update(current_state, request.is_correct)

    # Resolve scaffold level
    resolver = ScaffoldResolver()
    scaffold = resolver.resolve(new_state.p_mastery)

    # Check for misconception (3 consecutive wrong on same concept)
    misconception = None
    if new_state.consecutive_wrong >= 3:
        detector = MisconceptionDetector()
        misconception = await detector.classify(
            request.concept_id,
            current_state.response_history[-3:]
        )

    # Determine next action
    next_action = 'continue'
    if new_state.mastered:
```

```
        next_action = 'subtopic_complete'
    elif misconception:
        next_action = 'misconception_remediation'

    # Persist updated state to Firestore
    await fs.save_student_concept_state(new_state)

    # Save response record for analytics
    await fs.record_mcq_response(request, p_mastery_before, new_state.p_mastery)

    return BKTUpdateResponse(
        p_mastery_before=p_mastery_before,
        p_mastery_after=new_state.p_mastery,
        scaffold_level=scaffold.level,
        allowed_components=scaffold.allowed_components,
        mastered=new_state.mastered,
        misconception=misconception,
        next_action=next_action,
    )
```

8. Feature Specifications — Student Learning Experience

8.1 Topic Index with BKT-Driven Unlock Order

```
// apps/web/src/components/student/TopicIndex.tsx
'use client';

import { useEffect, useState } from 'react';
import { collection, onSnapshot, query, where } from 'firebase/firestore';
import { db } from '@/lib/firebase';
import { useBKTState } from '@/hooks/useBKTState';
import { Lock, CheckCircle, Circle, ChevronRight } from 'lucide-react';
import Link from 'next/link';

interface Subtopic {
  id: string;
  title: string;
  description: string;
  difficulty: 'foundational' | 'intermediate' | 'advanced';
  prerequisite_subtopic_ids: string[];
  order: number;
}

export function TopicIndex({ lessonId, studentId }: { lessonId: string; studentId: string }) {
  const [subtopics, setSubtopics] = useState<Subtopic[]>([]);
  const bktStates = useBKTState(studentId, lessonId);

  useEffect(() => {
    // Real-time subscription to subtopics
    const q = query(
      collection(db, 'lessons', lessonId, 'subtopics')
    );
    return onSnapshot(q, (snap) => {
      const data = snap.docs
        .map(d => ({ id: d.id, ...d.data() } as Subtopic))
        .sort((a, b) => a.order - b.order);
      setSubtopics(data);
    });
  }, [lessonId]);

  function getSubtopicStatus(subtopic: Subtopic): 'locked' | 'available' | 'in_progress' | 'mastered' {
    // Check prerequisites
    for (const prereqId of subtopic.prerequisite_subtopic_ids) {
      const prereqState = bktStates[prereqId];
      if (!prereqState || prereqState.p_mastery < 0.6) {
        return 'locked';
      }
    }

    const state = bktStates[subtopic.id];
    if (!state) return 'available';
    if (state.mastered) return 'mastered';
    if (state.attempts > 0) return 'in_progress';
    return 'available';
  }

  return (
    <nav className="w-72 bg-white border-r border-gray-200 h-full overflow-y-auto">
      <div className="p-4 border-b border-gray-200">
        <h2 className="font-semibold text-gray-900">Learning Path</h2>
        <div className="mt-2 bg-gray-100 rounded-full h-2">
          <div
            className="bg-green-500 h-2 rounded-full transition-all duration-500"
            style={{
              width: `${(Object.values(bktStates).filter(s => s.mastered).length / subtopics.length) * 100}%`
            }}
          />
        </div>
        <p className="text-xs text-gray-500 mt-1">
          {Object.values(bktStates).filter(s => s.mastered).length} of {subtopics.length} mastered
        </p>
      </div>

      <div className="p-2">
        {subtopics.map((subtopic) => {
```

```
          const status = getSubtopicStatus(subtopic);
          const state = bktStates[subtopic.id];

          return (
            <div key={subtopic.id} className="mb-1">
              {status === 'locked' ? (
                <div className="flex items-center gap-3 p-3 rounded-lg text-gray-400 cursor-not-allowed">
                  <Lock size={16} />
                  <div className="flex-1 min-w-0">
                    <p className="text-sm font-medium truncate">{subtopic.title}</p>
                    <p className="text-xs">
                      Complete prerequisites first
                    </p>
                  </div>
                </div>
              ) : (
                <Link
                  href={`/learn/${lessonId}/${subtopic.id}`}
                  className={`flex items-center gap-3 p-3 rounded-lg transition-colors
                    ${status === 'mastered'
                      ? 'bg-green-50 text-green-700 hover:bg-green-100'
                      : 'hover:bg-gray-100 text-gray-700'}`}
                >
                  {status === 'mastered'
                    ? <CheckCircle size={16} className="text-green-600 shrink-0" />
                    : status === 'in_progress'
                    ? <Circle size={16} className="text-blue-600 shrink-0 fill-blue-200" />
                    : <Circle size={16} className="text-gray-400 shrink-0" />
                  }
                  <div className="flex-1 min-w-0">
                    <p className="text-sm font-medium truncate">{subtopic.title}</p>
                    {state && !state.mastered && (
                      <div className="flex items-center gap-1 mt-0.5">
                        <div className="bg-gray-200 rounded-full h-1 flex-1">
                          <div
                            className="bg-blue-500 h-1 rounded-full"
                            style={{ width: `${state.p_mastery * 100}%` }}
                          />
                        </div>
                        <span className="text-xs text-gray-400">
                          {Math.round(state.p_mastery * 100)}%
                        </span>
                      </div>
                    )}
                  </div>
                  <ChevronRight size={14} className="shrink-0 text-gray-400" />
                </Link>
              )}
            </div>
          );
        })}
      </div>
    </nav>
  );
}
```

8.2 Adaptive MCQ Component

```
// apps/web/src/components/student/AdaptiveMCQ.tsx
'use client';

import { useState, useEffect } from 'react';
import { motion, AnimatePresence } from 'framer-motion';

interface MCQ {
  id: string;
  tier: 1 | 2 | 3;
  question: string;
  options: Record<'A' | 'B' | 'C' | 'D', string>;
  correct_answer: 'A' | 'B' | 'C' | 'D';
  concept: string;
  explanation: string;
  misconceptions: Record<string, string>;
}

interface AdaptiveMCQProps {
  question: MCQ;
  onAnswer: (answer: string, isCorrect: boolean, timeTaken: number) => Promise<void>;
  bktUpdateResult?: {
    p_mastery_before: number;
    p_mastery_after: number;
    misconception?: { type: string; explanation: string };
    next_action: string;
  };
}

export function AdaptiveMCQ({ question, onAnswer, bktUpdateResult }: AdaptiveMCQProps) {
  const [selected, setSelected] = useState<string | null>(null);
  const [revealed, setRevealed] = useState(false);
  const [startTime] = useState(Date.now());
  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleSelect = async (option: string) => {
    if (revealed || isSubmitting) return;

    setSelected(option);
    setIsSubmitting(true);

    const isCorrect = option === question.correct_answer;
    const timeTaken = Math.round((Date.now() - startTime) / 1000);

    await onAnswer(option, isCorrect, timeTaken);
    setRevealed(true);
    setIsSubmitting(false);
  };

  const optionColors = (option: string) => {
    if (!revealed) {
      return selected === option
        ? 'border-blue-500 bg-blue-50 text-blue-900'
        : 'border-gray-200 hover:border-gray-400 cursor-pointer';
    }
    if (option === question.correct_answer) return 'border-green-500 bg-green-50 text-green-900';
    if (option === selected && !revealed) return 'border-red-500 bg-red-50 text-red-900';
    if (option === selected && revealed && option !== question.correct_answer) {
      return 'border-red-400 bg-red-50 text-red-700';
    }
    return 'border-gray-200 text-gray-500';
  };

  return (
    <div className="space-y-4">
      {/* Tier badge */}
      <div className="flex items-center gap-2">
        <span className={`text-xs px-2 py-0.5 rounded-full font-medium
          ${question.tier === 1 ? 'bg-blue-100 text-blue-700' :
            question.tier === 2 ? 'bg-purple-100 text-purple-700' :
            'bg-orange-100 text-orange-700'}`}>
          {question.tier === 1 ? 'Foundation' : question.tier === 2 ? 'Understanding' : 'Analysis'}
        </span>
        <span className="text-xs text-gray-500">Testing: {question.concept}</span>
      </div>

      {/* Question */}
```

```
        <h3 className="text-base font-medium text-gray-900 leading-relaxed">
          {question.question}
        </h3>

        {/* Options */}
        <div className="space-y-2">
          {(Object.entries(question.options) as ['A' | 'B' | 'C' | 'D', string][]).map(([key, text]) => (
            <motion.button
              key={key}
              onClick={() => handleSelect(key)}
              className={`w-full text-left p-4 rounded-lg border-2 transition-all ${optionColors(key)}`}
              whileTap={{ scale: revealed ? 1 : 0.99 }}
              disabled={revealed}
            >
              <span className="font-semibold mr-2">{key}.</span>
              {text}
            </motion.button>
          ))}
        </div>

        {/* Feedback after answer */}
        <AnimatePresence>
          {revealed && (
            <motion.div
              initial={{ opacity: 0, y: 10 }}
              animate={{ opacity: 1, y: 0 }}
              className={`p-4 rounded-lg ${
                selected === question.correct_answer
                  ? 'bg-green-50 border border-green-200'
                  : 'bg-red-50 border border-red-200'
              }`}
            >
              {selected === question.correct_answer ? (
                <div>
                  <p className="font-medium text-green-800">✓ Correct!</p>
                  <p className="text-sm text-green-700 mt-1">{question.explanation}</p>
                </div>
              ) : (
                <div>
                  <p className="font-medium text-red-800">
                    Not quite — the correct answer is {question.correct_answer}
                  </p>
                  <p className="text-sm text-red-700 mt-1">
                    {question.misconceptions[selected!]}
                  </p>
                  <p className="text-sm text-gray-700 mt-2">{question.explanation}</p>
                </div>
              )}

              {/* BKT mastery delta */}
              {bktUpdateResult && (
                <div className="mt-3 pt-3 border-t border-gray-200">
                  <div className="flex items-center gap-2">
                    <span className="text-xs text-gray-500">Mastery</span>
                    <div className="flex-1 bg-gray-200 rounded-full h-1.5">
                      <div
                        className="bg-blue-500 h-1.5 rounded-full transition-all duration-700"
                        style={{ width: `${bktUpdateResult.p_mastery_after * 100}%` }}
                      />
                    </div>
                    <span className="text-xs text-gray-500">
                      {Math.round(bktUpdateResult.p_mastery_after * 100)}%
                    </span>
                  </div>
                </div>
              )}

              {/* Misconception alert */}
              {bktUpdateResult?.misconception && (
                <div className="mt-3 p-3 bg-amber-50 border border-amber-200 rounded-lg">
                  <p className="text-sm font-medium text-amber-800">
                    🤔 Let's clarify this misconception
                  </p>
                  <p className="text-sm text-amber-700 mt-1">
                    {bktUpdateResult.misconception.explanation}
                  </p>
                </div>
              )}
            </motion.div>
          )}
```

```
      </AnimatePresence>
    </div>
  );
}
```

8.3 Live Mastery HUD

```
// apps/web/src/components/student/MasteryHUD.tsx
'use client';

import { useBKTState } from '@/hooks/useBKTState';
import { useEffect, useRef } from 'react';
import * as d3 from 'd3';

export function MasteryHUD({
  studentId,
  lessonId,
  concepts
}: {
  studentId: string;
  lessonId: string;
  concepts: { id: string; label: string }[];
}) {
  const bktStates = useBKTState(studentId, lessonId);
  const svgRef = useRef<SVGSVGElement>(null);

  useEffect(() => {
    if (!svgRef.current || concepts.length === 0) return;

    const data = concepts.map(c => ({
      label: c.label,
      value: bktStates[c.id]?.p_mastery ?? 0.1,
      mastered: bktStates[c.id]?.mastered ?? false,
    }));

    renderRadarChart(svgRef.current, data);
  }, [bktStates, concepts]);

  return (
    <div className="bg-white rounded-xl border border-gray-200 p-4">
      <h3 className="text-sm font-semibold text-gray-700 mb-3">Concept Mastery</h3>
      <svg ref={svgRef} width="240" height="240" />
      <div className="mt-3 space-y-1.5">
        {concepts.map(c => {
          const state = bktStates[c.id];
          const p = state?.p_mastery ?? 0;
          return (
            <div key={c.id} className="flex items-center gap-2">
              <span className="text-xs text-gray-600 w-28 truncate">{c.label}</span>
              <div className="flex-1 bg-gray-100 rounded-full h-1.5">
                <div
                  className={`h-1.5 rounded-full transition-all duration-500 ${
                    state?.mastered ? 'bg-green-500' :
                    p > 0.6 ? 'bg-blue-500' :
                    p > 0.3 ? 'bg-yellow-500' : 'bg-red-400'
                  }`}
                  style={{ width: `${p * 100}%` }}
                />
              </div>
              <span className="text-xs text-gray-500 w-8 text-right">
                {Math.round(p * 100)}%
              </span>
            </div>
          );
        })}
      </div>
    </div>
  );
}

function renderRadarChart(svg: SVGSVGElement, data: { label: string; value: number; mastered: boolean }[]) {
  // D3 radar chart — each axis = one concept, radial position = P(mastery)
  const d3svg = d3.select(svg);
  d3svg.selectAll('*').remove();

  const width = 240, height = 240;
  const cx = width / 2, cy = height / 2;
  const r = 90;
  const n = data.length;

  // Draw grid circles
  [0.25, 0.5, 0.75, 1.0].forEach(level => {
    d3svg.append('circle')
```

```
      .attr('cx', cx).attr('cy', cy)
      .attr('r', r * level)
      .attr('fill', 'none')
      .attr('stroke', '#e5e7eb')
      .attr('stroke-width', 0.5);
  });

  // Draw axes
  data.forEach((_, i) => {
    const angle = (i * 2 * Math.PI / n) - Math.PI / 2;
    const x = cx + r * Math.cos(angle);
    const y = cy + r * Math.sin(angle);
    d3svg.append('line')
      .attr('x1', cx).attr('y1', cy)
      .attr('x2', x).attr('y2', y)
      .attr('stroke', '#d1d5db').attr('stroke-width', 0.5);

    // Labels
    const lx = cx + (r + 14) * Math.cos(angle);
    const ly = cy + (r + 14) * Math.sin(angle);
    d3svg.append('text')
      .attr('x', lx).attr('y', ly)
      .attr('text-anchor', 'middle')
      .attr('dominant-baseline', 'central')
      .attr('font-size', '9px')
      .attr('fill', '#6b7280')
      .text(data[i].label.length > 8 ? data[i].label.substring(0, 8) + '…' : data[i].label);
  });

  // Draw polygon
  const points = data.map((d, i) => {
    const angle = (i * 2 * Math.PI / n) - Math.PI / 2;
    const rv = r * d.value;
    return [cx + rv * Math.cos(angle), cy + rv * Math.sin(angle)];
  });

  d3svg.append('polygon')
    .attr('points', points.map(p => p.join(',')).join(' '))
    .attr('fill', 'rgba(59, 130, 246, 0.15)')
    .attr('stroke', '#3b82f6')
    .attr('stroke-width', 1.5);

  // Dots
  points.forEach((p, i) => {
    d3svg.append('circle')
      .attr('cx', p[0]).attr('cy', p[1]).attr('r', 3)
      .attr('fill', data[i].mastered ? '#22c55e' : '#3b82f6');
  });
}
```

9. Feature Specifications — GenUI Layer

9.1 OpenUI Component Library Definition

```
// apps/web/src/components/genui/library.ts
import { z } from 'zod';
import { defineComponent, createLibrary } from '@openuidev/react-lang';

// ── COMPONENT DEFINITIONS ──────────────────────────────────────────────

const StepByStep = defineComponent({
  name: 'StepByStep',
  description: 'Numbered step-by-step guide for beginners. Use when P(mastery) < 0.4.',
  props: z.object({
    concept: z.string().describe('The concept being explained'),
    steps: z.array(z.object({
      number: z.number(),
      title: z.string(),
      explanation: z.string(),
      example: z.string().optional(),
    })),
    summary: z.string().optional(),
  }),
  component: StepByStepComponent,
});

const HintCard = defineComponent({
  name: 'HintCard',
  description: 'Provides a hint or partial explanation. Use for scaffold levels 0-2.',
  props: z.object({
    hint_level: z.enum(['gentle', 'moderate', 'direct']),
    hint_text: z.string(),
    follow_up_question: z.string().optional().describe('Socratic follow-up question after hint'),
  }),
  component: HintCardComponent,
});

const FormulaCard = defineComponent({
  name: 'FormulaCard',
  description: 'Displays a formula or equation with explanation. Always available as reference.',
  props: z.object({
    formula: z.string().describe('The formula in plain text or LaTeX'),
    variables: z.array(z.object({
      symbol: z.string(),
      name: z.string(),
      unit: z.string().optional(),
    })),
    example: z.object({
      values: z.record(z.string()),
      result: z.string(),
    }).optional(),
  }),
  component: FormulaCardComponent,
});

const ConceptDiagram = defineComponent({
  name: 'ConceptDiagram',
  description: 'Visual diagram explaining a concept. Use for scaffold levels 2+.',
  props: z.object({
    title: z.string(),
    diagram_type: z.enum(['process_flow', 'comparison', 'hierarchy', 'cycle', 'relationship']),
    elements: z.array(z.object({
      id: z.string(),
      label: z.string(),
      description: z.string().optional(),
      connects_to: z.array(z.string()).optional(),
    })),
    annotations: z.array(z.string()).optional(),
  }),
  component: ConceptDiagramComponent,
});

const AnalogyCard = defineComponent({
  name: 'AnalogyCard',
  description: 'Real-world analogy for abstract concepts. Best for scaffold level 0-1.',
```

```
  props: z.object({
    abstract_concept: z.string(),
    real_world_analogy: z.string(),
    how_they_match: z.array(z.object({
      concept_aspect: z.string(),
      analogy_aspect: z.string(),
    })),
    limitation: z.string().optional().describe('Where the analogy breaks down'),
  }),
  component: AnalogyCardComponent,
});

const PracticeExercise = defineComponent({
  name: 'PracticeExercise',
  description: 'Practice exercise with immediate feedback. For scaffold levels 2+.',
  props: z.object({
    problem: z.string(),
    hints: z.array(z.string()).optional(),
    worked_solution: z.string(),
    key_insight: z.string(),
  }),
  component: PracticeExerciseComponent,
});

const ProofWalkthrough = defineComponent({
  name: 'ProofWalkthrough',
  description: 'Mathematical proof walkthrough. For scaffold levels 3+.',
  props: z.object({
    theorem: z.string(),
    proof_steps: z.array(z.object({
      step: z.number(),
      statement: z.string(),
      justification: z.string(),
    })),
    conclusion: z.string(),
  }),
  component: ProofWalkthroughComponent,
});

const ExpertSummary = defineComponent({
  name: 'ExpertSummary',
  description: 'Dense expert-level summary. Only for scaffold level 4 (mastered).',
  props: z.object({
    key_ideas: z.array(z.string()),
    common_pitfalls: z.array(z.string()),
    advanced_connections: z.array(z.string()),
    challenge_question: z.string(),
  }),
  component: ExpertSummaryComponent,
});

// ── LIBRARY ──────────────────────────────────────────────────────────

export const FULL_COMPONENT_LIBRARY = createLibrary({
  root: 'StepByStep',
  components: [
    StepByStep, HintCard, FormulaCard, ConceptDiagram,
    AnalogyCard, PracticeExercise, ProofWalkthrough, ExpertSummary
  ],
});

// Gated libraries per scaffold level (matches scaffold_resolver.py)
export const GATED_LIBRARIES: Record<number, ReturnType<typeof createLibrary>> = {
  0: createLibrary({ root: 'StepByStep', components: [StepByStep, HintCard, FormulaCard, AnalogyCard] }),
  1: createLibrary({ root: 'StepByStep', components: [StepByStep, HintCard, FormulaCard, ConceptDiagram] }),
  2: createLibrary({ root: 'ConceptDiagram', components: [ConceptDiagram, FormulaCard, HintCard, PracticeExercise] }),
  3: createLibrary({ root: 'ConceptDiagram', components: [ConceptDiagram, PracticeExercise, ProofWalkthrough] }),
  4: createLibrary({ root: 'ConceptDiagram', components: [ConceptDiagram, ExpertSummary, ProofWalkthrough, PracticeExercise] }),
};
```

9.2 GenUI Streaming Service

```python
# apps/genui-service/src/genui/streamer.py
import anthropic
import json
import os
from .prompt_builder import GenUIPromptBuilder
from .catalog import get_catalog_for_scaffold_level

client = anthropic.Anthropic(api_key=os.environ['CLAUDE_API_KEY'])

class GenUIStreamer:
    """
    Streams a BKT-constrained GenUI visualization for a concept.
    The BKT scaffold level determines which OpenUI components the LLM can use.

    This is the core neuro-symbolic fusion:
    Symbolic BKT posterior → component catalog constraint → Neural LLM generation
    """

    def stream_visualization(
        self,
        concept: str,
        concept_content: str,
        bkt_state: dict,  # From BKT service
        scaffold_decision: dict,  # From scaffold resolver
        student_name: str = "the student"
    ):
        """
        Yields streaming GenUI JSON tokens.
        The component catalog passed to the LLM is determined by bkt_state.p_mastery.
        """

        prompt_builder = GenUIPromptBuilder()
        system_prompt = prompt_builder.build(
            allowed_components=scaffold_decision['allowed_components'],
            scaffold_level=scaffold_decision['level'],
            p_mastery=bkt_state['p_mastery'],
        )

        user_message = f"""Create a learning visualization for this concept:

CONCEPT: {concept}

CONTENT FROM LESSON:
{concept_content}

STUDENT STATE:
- Mastery: {bkt_state['p_mastery']:.2f} ({scaffold_decision['level_name']})
- Attempts: {bkt_state.get('attempts', 0)}
- Last answer correct: {bkt_state.get('last_correct', None)}

Generate the most appropriate visualization using ONLY the allowed components.
Tailor the depth, complexity, and language to the student's mastery level."""

        # Stream from Claude API
        with client.messages.stream(
            model="claude-sonnet-4-6",
            max_tokens=2000,
            system=system_prompt,
            messages=[{"role": "user", "content": user_message}],
        ) as stream:
            for text in stream.text_stream:
                yield text

    def stream_tutor_response(
        self,
        student_message: str,
        concept: str,
        bkt_state: dict,
        scaffold_decision: dict,
        conversation_history: list,
    ):
        """
        Streams a Socratic tutor response constrained by BKT scaffold level.
        At high mastery: ask leading questions, don't give answers.
        At low mastery: explain clearly, provide scaffold.
        """
```

```
        level = scaffold_decision['level']
        p_mastery = bkt_state['p_mastery']

        tutor_system = f"""You are an expert tutor helping a student understand: {concept}

{scaffold_decision['system_prompt_fragment']}

BEHAVIORAL RULES:
{"- DO NOT give direct answers. Ask Socratic leading questions." if level >= 4 else ""}
{"- Guide with hints. Never overwhelm. One step at a time." if level <= 1 else ""}
{"- Use simple language. Avoid jargon. Use analogies." if level == 0 else ""}
- Be encouraging and supportive at all times.
- Keep responses concise (3-5 sentences max unless explanation requires more).
- If student is confused, reframe the concept from a different angle.

FORBIDDEN: Do not mention P(mastery), BKT, scaffold levels, or any system internals."""

        messages = conversation_history + [
            {"role": "user", "content": student_message}
        ]

        with client.messages.stream(
            model="claude-sonnet-4-6",
            max_tokens=800,
            system=tutor_system,
            messages=messages,
        ) as stream:
            for text in stream.text_stream:
                yield text
```

9.3 GenUI API Route (Next.js Streaming)

```
// apps/web/src/app/api/genui/route.ts
import { NextRequest } from 'next/server';

export const runtime = 'edge'; // Use Edge runtime for streaming

export async function POST(req: NextRequest) {
  const {
    conceptId,
    subtopicId,
    lessonId,
    studentId,
    type // 'visualize' | 'chat'
  } = await req.json();

  // 1. Get current BKT state from BKT service
  const bktResponse = await fetch(
    `${process.env.BKT_SERVICE_URL}/state?studentId=${studentId}&conceptId=${conceptId}`
  );
  const bktState = await bktResponse.json();

  // 2. Get scaffold decision
  const scaffoldResponse = await fetch(
    `${process.env.BKT_SERVICE_URL}/scaffold?p_mastery=${bktState.p_mastery}`
  );
  const scaffoldDecision = await scaffoldResponse.json();

  // 3. Stream from GenUI service
  const genUIResponse = await fetch(`${process.env.GENUI_SERVICE_URL}/stream`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ bktState, scaffoldDecision, conceptId, lessonId, type }),
  });

  // 4. Pass through the stream to the client
  return new Response(genUIResponse.body, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive',
    },
  });
}
```

10. Feature Specifications — Analytics Dashboards

10.1 Teacher Class Heatmap

```
// apps/web/src/components/teacher/ClassHeatmap.tsx
/**
 * Real-time heatmap of student mastery per concept.
 * Teacher sees:
 * - Grid: rows = students, columns = subtopics
 * - Cell color: green (mastered) → yellow (approaching) → red (struggling)
 * - Click cell: drill down to individual student+concept view
 * - Alerts: concepts where >50% of class is stuck
 */

// Firestore query aggregates BKT states across all students
// onSnapshot listener updates heatmap in real-time as students answer MCQs
```

10.2 Student Session Analytics

Track per-session:

- Time spent per subtopic
- MCQ accuracy per tier (Tier 1/2/3)
- BKT mastery trajectory (line chart over time)
- Misconception frequency by concept
- Average scaffold level over time (should decrease as student learns)

11. Data Models & Firestore Schema

11.1 Firestore Collections

```
firestore/
├── users/{userId}
│   ├── uid: string
│   ├── email: string
│   ├── displayName: string
│   ├── role: 'teacher' | 'student'
│   ├── institution: string
│   └── createdAt: timestamp
│
├── lessons/{lessonId}
│   ├── title: string
│   ├── subject: string
│   ├── teacherId: string
│   ├── status: 'draft' | 'processing' | 'published' | 'archived'
│   ├── gcsPath: string
│   ├── createdAt: timestamp
│   ├── publishedAt: timestamp?
│   └── ingestion: {
│       ├── step: IngestionStep
│       ├── progress: number (0-100)
│       ├── message: string
│       ├── subtopicsFound: number?
│       ├── mcqsGenerated: number?
│       └── error: string?
│   }
│
├── lessons/{lessonId}/subtopics/{subtopicId}
│   ├── title: string
│   ├── description: string
│   ├── order: number
│   ├── difficulty: 'foundational' | 'intermediate' | 'advanced'
│   ├── slideNumbers: number[]
│   ├── keyConcepts: string[]
│   ├── learningObjectives: string[]
│   └── prerequisiteSubtopicIds: string[]
│
├── lessons/{lessonId}/mcqs/{mcqId}
│   ├── subtopicId: string
│   ├── conceptId: string
│   ├── tier: 1 | 2 | 3
│   ├── question: string
│   ├── options: { A: string, B: string, C: string, D: string }
│   ├── correctAnswer: 'A' | 'B' | 'C' | 'D'
│   ├── explanation: string
│   ├── misconceptions: Record<string, string>
│   └── concept: string
│
├── lessons/{lessonId}/bkt_params/{conceptId}
│   ├── conceptId: string
│   ├── conceptLabel: string
│   ├── subtopicId: string
│   ├── pInitial: number (0.2)
│   ├── pLearn: number (0.2)
│   ├── pSlip: number (0.1)
│   └── pGuess: number (0.25)
│
├── enrollments/{enrollmentId}
│   ├── studentId: string
│   ├── lessonId: string
│   ├── enrolledAt: timestamp
│   └── status: 'active' | 'completed'
│
├── bkt_states/{studentId}/concepts/{conceptId}
│   ├── studentId: string
│   ├── lessonId: string
│   ├── subtopicId: string
│   ├── conceptId: string
│   ├── pMastery: number
│   ├── attempts: number
│   ├── correctStreak: number
│   ├── mastered: boolean
```

```
│   ├── consecutiveWrong: number
│   ├── lastUpdated: timestamp
│   └── responseHistory: Array<{isCorrect, pMasteryBefore, pMasteryAfter}>
│
└── responses/{responseId}
    ├── studentId: string
    ├── lessonId: string
    ├── subtopicId: string
    ├── conceptId: string
    ├── mcqId: string
    ├── selectedAnswer: string
    ├── isCorrect: boolean
    ├── pMasteryBefore: number
    ├── pMasteryAfter: number
    ├── scaffoldLevel: number
    ├── timeTakenSeconds: number
    └── answeredAt: timestamp
```

11.2 Firestore Security Rules

```
// firestore.rules
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Helper functions
    function isAuthenticated() {
      return request.auth != null;
    }

    function isTeacher() {
      return isAuthenticated() &&
        get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role == 'teacher';
    }

    function isStudent() {
      return isAuthenticated() &&
        get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role == 'student';
    }

    function ownsLesson(lessonId) {
      return get(/databases/$(database)/documents/lessons/$(lessonId)).data.teacherId == request.auth.uid;
    }

    function isEnrolled(lessonId) {
      return exists(/databases/$(database)/documents/enrollments/$(request.auth.uid + '_' + lessonId));
    }

    // User profiles
    match /users/{userId} {
      allow read: if isAuthenticated();
      allow write: if request.auth.uid == userId;
    }

    // Lessons — teacher can CRUD, students can read published
    match /lessons/{lessonId} {
      allow read: if isAuthenticated() && (ownsLesson(lessonId) || isEnrolled(lessonId));
      allow create: if isTeacher();
      allow update, delete: if isTeacher() && ownsLesson(lessonId);

      match /subtopics/{subtopicId} {
        allow read: if isAuthenticated() && (ownsLesson(lessonId) || isEnrolled(lessonId));
        allow write: if isTeacher() && ownsLesson(lessonId);
      }

      match /mcqs/{mcqId} {
        allow read: if isAuthenticated() && (ownsLesson(lessonId) || isEnrolled(lessonId));
        allow write: if isTeacher() && ownsLesson(lessonId);
      }

      match /bkt_params/{conceptId} {
        allow read: if isAuthenticated();
        allow write: if isTeacher() && ownsLesson(lessonId);
      }
    }

    // BKT states — student can read/write their own, teacher can read all
    match /bkt_states/{studentId}/concepts/{conceptId} {
      allow read: if request.auth.uid == studentId || isTeacher();
      allow write: if request.auth.uid == studentId;
    }

    // Responses — student writes own, teacher reads all for their lessons
    match /responses/{responseId} {
      allow create: if isAuthenticated() && request.resource.data.studentId == request.auth.uid;
      allow read: if isAuthenticated() && (
        resource.data.studentId == request.auth.uid || isTeacher()
      );
    }

    // Enrollments
    match /enrollments/{enrollmentId} {
      allow read: if isAuthenticated();
      allow create: if isStudent();
    }
```

```
  }
}
```

11.3 Firestore Indexes

```
// firestore.indexes.json
{
  "indexes": [
    {
      "collectionGroup": "responses",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "studentId", "order": "ASCENDING" },
        { "fieldPath": "lessonId", "order": "ASCENDING" },
        { "fieldPath": "answeredAt", "order": "DESCENDING" }
      ]
    },
    {
      "collectionGroup": "responses",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "lessonId", "order": "ASCENDING" },
        { "fieldPath": "conceptId", "order": "ASCENDING" },
        { "fieldPath": "answeredAt", "order": "DESCENDING" }
      ]
    },
    {
      "collectionGroup": "bkt_states",
      "queryScope": "COLLECTION_GROUP",
      "fields": [
        { "fieldPath": "lessonId", "order": "ASCENDING" },
        { "fieldPath": "pMastery", "order": "ASCENDING" }
      ]
    }
  ]
}
```

---

12. API Contracts

12.1 API Gateway (FastAPI — Cloud Run)

```
Base URL: https://api-{PROJECT_ID}-uc.a.run.app

Authentication: Bearer {Firebase ID Token} in Authorization header

POST   /lessons/upload-url          → { uploadUrl, lessonId, gcsPath }
POST   /lessons/start-ingestion     → { jobId, status: "queued" }
GET    /lessons/{lessonId}          → Lesson details + ingestion status
GET    /lessons/{lessonId}/subtopics → List of subtopics
GET    /lessons/{lessonId}/subtopics/{subtopicId}/mcqs → MCQ bank for subtopic
POST   /enrollments                 → Enroll student in lesson
GET    /students/{studentId}/lessons → List enrolled lessons
GET    /analytics/class/{lessonId}  → Class-level BKT aggregates (teacher only)
POST   /internal/lesson-complete    → Pub/Sub callback (internal only)
```

12.2 BKT Service

```
Base URL: https://bkt-{PROJECT_ID}-uc.a.run.app

POST   /update                      → BKT update after MCQ answer
GET    /state?studentId=&conceptId= → Current BKT state
GET    /scaffold?p_mastery=         → Scaffold level + allowed components
GET    /lesson-states?studentId=&lessonId= → All concept states for a lesson
```

12.3 GenUI Service

```
Base URL: https://genui-{PROJECT_ID}-uc.a.run.app

POST   /stream/visualize            → SSE stream of GenUI component JSON
POST   /stream/chat                 → SSE stream of AI tutor response
```

---

13. Frontend Architecture

13.1 Next.js Configuration

```python
// apps/web/next.config.ts
import type { NextConfig } from 'next';

const nextConfig: NextConfig = {
  experimental: {
    serverActions: { allowedOrigins: ['localhost:3000', process.env.NEXT_PUBLIC_APP_URL!] },
  },
  images: {
    remotePatterns: [{
      protocol: 'https',
      hostname: 'storage.googleapis.com',
    }],
  },
  env: {
    NEXT_PUBLIC_FIREBASE_PROJECT_ID: process.env.FIREBASE_PROJECT_ID,
    NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN: process.env.FIREBASE_AUTH_DOMAIN,
    NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET: process.env.FIREBASE_STORAGE_BUCKET,
    NEXT_PUBLIC_APP_URL: process.env.NEXT_PUBLIC_APP_URL,
  },
};

export default nextConfig;
```

13.2 BKT State Zustand Store

```
// apps/web/src/stores/bktStore.ts
import { create } from 'zustand';
import { subscribeWithSelector } from 'zustand/middleware';
import { collection, onSnapshot, query } from 'firebase/firestore';
import { db } from '@/lib/firebase';

interface ConceptState {
  conceptId: string;
  pMastery: number;
  mastered: boolean;
  attempts: number;
  scaffoldLevel: number;
}

interface BKTStore {
  states: Record<string, ConceptState>;  // conceptId → state
  scaffoldLevel: number;                  // Current scaffold level for active concept
  allowedComponents: string[];            // Current allowed GenUI components

  updateConceptState: (conceptId: string, state: Partial<ConceptState>) => void;
  setScaffoldDecision: (level: number, components: string[]) => void;
  subscribeToLesson: (studentId: string, lessonId: string) => () => void;
}

export const useBKTStore = create<BKTStore>()(
  subscribeWithSelector((set) => ({
    states: {},
    scaffoldLevel: 0,
    allowedComponents: ['StepByStep', 'HintCard', 'FormulaCard', 'AnalogyCard'],

    updateConceptState: (conceptId, state) =>
      set(s => ({
        states: { ...s.states, [conceptId]: { ...s.states[conceptId], ...state } }
      })),

    setScaffoldDecision: (level, components) =>
      set({ scaffoldLevel: level, allowedComponents: components }),

    subscribeToLesson: (studentId, lessonId) => {
      // Real-time Firestore listener for all BKT states in this lesson
      const q = query(
        collection(db, 'bkt_states', studentId, 'concepts')
      );

      return onSnapshot(q, (snap) => {
        const states: Record<string, ConceptState> = {};
        snap.docs.forEach(doc => {
          const data = doc.data();
          if (data.lessonId === lessonId) {
            states[doc.id] = {
              conceptId: doc.id,
              pMastery: data.pMastery,
              mastered: data.mastered,
              attempts: data.attempts,
              scaffoldLevel: Math.floor(data.pMastery / 0.2),
            };
          }
        });
        set({ states });
      });
    },
  }))
);
```

14. Backend Services Architecture

14.1 FastAPI Base Middleware

```python
# apps/api/main.py
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
import time
import logging

from src.routers import lessons, students, bkt, analytics, internal
from src.middleware.auth import verify_firebase_token

app = FastAPI(
    title="EduForge API",
    version="1.0.0",
    docs_url="/docs",
)

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",
        f"https://{os.environ.get('NEXT_PUBLIC_APP_URL', '')}",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Request timing middleware
@app.middleware("http")
async def add_timing(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    response.headers["X-Process-Time"] = str(round((time.time() - start) * 1000, 2)) + "ms"
    return response

# Include routers
app.include_router(lessons.router, prefix="/lessons", tags=["lessons"])
app.include_router(students.router, prefix="/students", tags=["students"])
app.include_router(bkt.router, prefix="/bkt", tags=["bkt"])
app.include_router(analytics.router, prefix="/analytics", tags=["analytics"])
app.include_router(internal.router, prefix="/internal", tags=["internal"])

@app.get("/health")
async def health():
    return {"status": "healthy", "service": "eduforge-api"}
```

14.2 Firebase Auth Middleware

```python
# apps/api/src/middleware/auth.py
from fastapi import HTTPException, Security
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import firebase_admin
from firebase_admin import auth, credentials
import os
import json

# Initialize Firebase Admin
if not firebase_admin._apps:
    sa_json = os.environ.get('FIREBASE_SERVICE_ACCOUNT_JSON')
    if sa_json:
        cred = credentials.Certificate(json.loads(sa_json))
    else:
        cred = credentials.ApplicationDefault()
    firebase_admin.initialize_app(cred)

security = HTTPBearer()

async def verify_firebase_token(
    credentials: HTTPAuthorizationCredentials = Security(security)
) -> dict:
    """Verify Firebase ID token and return decoded claims."""
    try:
        decoded = auth.verify_id_token(credentials.credentials)
        return decoded
    except Exception as e:
        raise HTTPException(status_code=401, detail=f"Invalid token: {str(e)}")

async def require_teacher(token: dict = Security(verify_firebase_token)) -> dict:
    """Require teacher role."""
    # Fetch user profile to check role
    # In practice, store role as custom claim in Firebase Auth
    return token

async def require_student(token: dict = Security(verify_firebase_token)) -> dict:
    """Require student role."""
    return token
```

---

15. Authentication & Security

15.1 Firebase Auth Setup

```typescript
// apps/web/src/lib/firebase.ts
import { initializeApp, getApps } from 'firebase/app';
import { getAuth } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';
import { getStorage } from 'firebase/storage';

const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID,
};

const app = getApps().length === 0 ? initializeApp(firebaseConfig) : getApps()[0];

export const auth = getAuth(app);
export const db = getFirestore(app);
export const storage = getStorage(app);
```

15.2 Role-Based Route Protection

```
// apps/web/src/middleware.ts
import { NextRequest, NextResponse } from 'next/server';

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Teacher routes require teacher role (checked via cookie/session)
  if (pathname.startsWith('/dashboard') || pathname.startsWith('/lessons/new')) {
    const role = request.cookies.get('user-role')?.value;
    if (role !== 'teacher') {
      return NextResponse.redirect(new URL('/login', request.url));
    }
  }

  // Student routes require student role
  if (pathname.startsWith('/learn')) {
    const role = request.cookies.get('user-role')?.value;
    if (role !== 'student') {
      return NextResponse.redirect(new URL('/login', request.url));
    }
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*', '/learn/:path*', '/lessons/:path*'],
};
```

16. Testing Strategy

16.1 BKT Engine Unit Tests

```python
# apps/bkt-service/tests/test_bkt_engine.py
import pytest
from src.bkt.engine import BKTEngine, BKTParams, StudentConceptState

@pytest.fixture
def default_params():
    return BKTParams(
        concept_id="test_concept",
        p_initial=0.2,
        p_learn=0.2,
        p_slip=0.1,
        p_guess=0.25,
    )

@pytest.fixture
def initial_state(default_params):
    return StudentConceptState(
        student_id="student_1",
        concept_id="test_concept",
        subtopic_id="subtopic_1",
        p_mastery=default_params.p_initial,
    )

class TestBKTEngine:
    def test_correct_answer_increases_mastery(self, default_params, initial_state):
        engine = BKTEngine(default_params)
        new_state = engine.update(initial_state, is_correct=True)
        assert new_state.p_mastery > initial_state.p_mastery

    def test_wrong_answer_decreases_mastery(self, default_params, initial_state):
        engine = BKTEngine(default_params)
        new_state = engine.update(initial_state, is_correct=False)
        assert new_state.p_mastery < initial_state.p_mastery

    def test_mastery_reaches_threshold_after_consecutive_correct(self, default_params):
        """After many correct answers, mastery should approach threshold."""
        engine = BKTEngine(default_params)
        state = StudentConceptState(
            student_id="s1", concept_id="c1", subtopic_id="st1", p_mastery=0.2
        )
        for _ in range(20):
            state = engine.update(state, is_correct=True)
        assert state.p_mastery >= 0.8  # Should be high after 20 correct

    def test_mastery_stays_bounded(self, default_params, initial_state):
        """P(mastery) should always be in (0, 1)."""
        engine = BKTEngine(default_params)
        state = initial_state
        for i in range(50):
            state = engine.update(state, is_correct=i % 3 != 0)
        assert 0 < state.p_mastery < 1

    def test_streak_tracking(self, default_params, initial_state):
        engine = BKTEngine(default_params)
        state = engine.update(initial_state, is_correct=True)
        assert state.correct_streak == 1
        state = engine.update(state, is_correct=True)
        assert state.correct_streak == 2
        state = engine.update(state, is_correct=False)
        assert state.correct_streak == 0


class TestScaffoldResolver:
    def test_level_0_for_low_mastery(self):
        from src.bkt.scaffold_resolver import ScaffoldResolver
        resolver = ScaffoldResolver()
        decision = resolver.resolve(0.1)
        assert decision.level == 0
        assert 'StepByStep' in decision.allowed_components

    def test_level_4_for_high_mastery(self):
```

```
        from src.bkt.scaffold_resolver import ScaffoldResolver
        resolver = ScaffoldResolver()
        decision = resolver.resolve(0.9)
        assert decision.level == 4
        assert 'ExpertSummary' in decision.allowed_components
        assert 'HintCard' not in decision.allowed_components  # Not in level 4

    def test_components_increase_with_mastery(self):
        from src.bkt.scaffold_resolver import ScaffoldResolver
        resolver = ScaffoldResolver()
        d0 = resolver.resolve(0.1)
        d4 = resolver.resolve(0.9)
        # Higher mastery → different (more advanced) components
        assert set(d0.allowed_components) != set(d4.allowed_components)
```

16.2 Integration Test — Full MCQ Answer Flow

```
# apps/api/tests/test_integration.py
import pytest
from fastapi.testclient import TestClient
from unittest.mock import patch, AsyncMock

@pytest.mark.asyncio
async def test_full_mcq_answer_flow():
    """
    Tests the full flow:
    1. Student answers MCQ
    2. BKT updates
    3. Scaffold level changes
    4. Allowed components change
    5. Firestore state persists
    """
    # ... test implementation
```

16.3 Frontend Component Tests

```
// apps/web/src/components/student/__tests__/AdaptiveMCQ.test.tsx
import { render, screen, fireEvent, waitFor } from '@testing-library/react';
import { AdaptiveMCQ } from '../AdaptiveMCQ';

const mockQuestion = {
  id: 'mcq_1',
  tier: 1 as const,
  question: "What is Newton's First Law?",
  options: {
    A: "Objects in motion stay in motion",
    B: "Force equals mass times acceleration",
    C: "Every action has an equal reaction",
    D: "Objects always slow down",
  },
  correct_answer: 'A' as const,
  concept: 'Inertia',
  explanation: 'Newton\'s First Law states that objects in motion stay in motion...',
  misconceptions: {
    B: 'That is Newton\'s Second Law...',
    C: 'That is Newton\'s Third Law...',
    D: 'Objects only slow down if a force acts on them...',
  }
};

describe('AdaptiveMCQ', () => {
  it('renders question and all 4 options', () => {
    render(<AdaptiveMCQ question={mockQuestion} onAnswer={jest.fn()} />);
    expect(screen.getByText(mockQuestion.question)).toBeInTheDocument();
    expect(screen.getByText('A.')).toBeInTheDocument();
  });

  it('calls onAnswer with correct isCorrect flag', async () => {
    const onAnswer = jest.fn().mockResolvedValue(undefined);
    render(<AdaptiveMCQ question={mockQuestion} onAnswer={onAnswer} />);

    fireEvent.click(screen.getByText('A.'));

    await waitFor(() => {
      expect(onAnswer).toHaveBeenCalledWith('A', true, expect.any(Number));
    });
  });

  it('shows explanation after answering', async () => {
    const onAnswer = jest.fn().mockResolvedValue(undefined);
    render(<AdaptiveMCQ question={mockQuestion} onAnswer={onAnswer} />);

    fireEvent.click(screen.getByText('A.'));

    await waitFor(() => {
      expect(screen.getByText(/Correct!/)).toBeInTheDocument();
    });
  });
});
```

16.4 End-to-End Test Script

```
#!/bin/bash
# scripts/test/run_all_tests.sh

set -e

echo "=== Running EduForge Test Suite ==="

# Python services
echo "--- BKT Service Tests ---"
cd apps/bkt-service
pip install -r requirements.txt -q
python -m pytest tests/ -v --tb=short
cd ../..

echo "--- API Service Tests ---"
cd apps/api
python -m pytest tests/ -v --tb=short
cd ../..

echo "--- Ingestion Service Tests ---"
cd apps/ingestion
python -m pytest tests/ -v --tb=short
cd ../..

# Frontend
echo "--- Frontend Tests ---"
cd apps/web
npm ci --silent
npm test -- --watchAll=false --passWithNoTests
cd ../..

echo "=== ✅ All tests passed ==="
```

---

17. Deployment Scripts & CI/CD

17.1 Build & Push Docker Images

```bash
#!/bin/bash
# scripts/gcp/build_and_push.sh

set -e

PROJECT_ID=$(gcloud config get-value project)
REGION="us-central1"
REGISTRY="${REGION}-docker.pkg.dev/${PROJECT_ID}/eduforge"
TAG="${1:-latest}"

echo "Building and pushing images with tag: $TAG"

# Configure Docker auth
gcloud auth configure-docker ${REGION}-docker.pkg.dev --quiet

# Build and push each service
services=("api" "bkt-service" "ingestion" "genui-service")

for service in "${services[@]}"; do
  echo "--- Building ${service} ---"
  docker build \
    --platform linux/amd64 \
    -t "${REGISTRY}/${service}:${TAG}" \
    -f "apps/${service}/Dockerfile" \
    "apps/${service}"

  docker push "${REGISTRY}/${service}:${TAG}"
  echo "✅ ${service} pushed"
done

echo "=== All images pushed to ${REGISTRY} ==="
```

17.2 Deploy All Services

```bash
#!/bin/bash
# scripts/gcp/deploy_all.sh

set -e

PROJECT_ID=$(gcloud config get-value project)
REGION="us-central1"
REGISTRY="${REGION}-docker.pkg.dev/${PROJECT_ID}/eduforge"
TAG="${1:-latest}"

API_SA="eduforge-api@${PROJECT_ID}.iam.gserviceaccount.com"
INGESTION_SA="eduforge-ingestion@${PROJECT_ID}.iam.gserviceaccount.com"
BKT_SA="eduforge-bkt@${PROJECT_ID}.iam.gserviceaccount.com"
GENUI_SA="eduforge-genui@${PROJECT_ID}.iam.gserviceaccount.com"

echo "🚀 Deploying EduForge to GCP project: $PROJECT_ID"

# ── DEPLOY API SERVICE ──────────────────────────────────────────
echo "--- Deploying API Service ---"
gcloud run deploy eduforge-api \
  --image="${REGISTRY}/api:${TAG}" \
  --region=$REGION \
  --service-account=$API_SA \
  --allow-unauthenticated \
  --memory=512Mi \
  --cpu=1 \
  --min-instances=0 \
  --max-instances=10 \
  --set-env-vars="GOOGLE_CLOUD_PROJECT=${PROJECT_ID}" \
  --set-secrets="CLAUDE_API_KEY=CLAUDE_API_KEY:latest,GEMINI_API_KEY=GEMINI_API_KEY:latest,FIREBASE_SERVICE_ACCOUNT=FIREBASE_SERVICE_AC
  --project=$PROJECT_ID

API_URL=$(gcloud run services describe eduforge-api \
  --region=$REGION \
  --format="value(status.url)" \
  --project=$PROJECT_ID)
echo "✅ API Service: $API_URL"

# ── DEPLOY BKT SERVICE ──────────────────────────────────────────
echo "--- Deploying BKT Service ---"
gcloud run deploy eduforge-bkt \
  --image="${REGISTRY}/bkt-service:${TAG}" \
  --region=$REGION \
  --service-account=$BKT_SA \
  --no-allow-unauthenticated \
  --memory=512Mi \
  --cpu=1 \
  --min-instances=0 \
  --max-instances=20 \
  --set-env-vars="GOOGLE_CLOUD_PROJECT=${PROJECT_ID}" \
  --project=$PROJECT_ID

BKT_URL=$(gcloud run services describe eduforge-bkt \
  --region=$REGION \
  --format="value(status.url)" \
  --project=$PROJECT_ID)
echo "✅ BKT Service: $BKT_URL"

# ── DEPLOY GENUI SERVICE ────────────────────────────────────────
echo "--- Deploying GenUI Service ---"
gcloud run deploy eduforge-genui \
  --image="${REGISTRY}/genui-service:${TAG}" \
  --region=$REGION \
  --service-account=$GENUI_SA \
  --no-allow-unauthenticated \
  --memory=512Mi \
  --cpu=1 \
  --min-instances=0 \
  --max-instances=10 \
  --timeout=300 \
  --set-secrets="CLAUDE_API_KEY=CLAUDE_API_KEY:latest" \
  --set-env-vars="GOOGLE_CLOUD_PROJECT=${PROJECT_ID}" \
  --project=$PROJECT_ID

GENUI_URL=$(gcloud run services describe eduforge-genui \
  --region=$REGION \
```

```
  --format="value(status.url)" \
  --project=$PROJECT_ID)
echo "✅ GenUI Service: $GENUI_URL"

# ── DEPLOY INGESTION TRIGGER SERVICE ──────────────────────────────────
echo "--- Deploying Ingestion Trigger Service ---"
gcloud run deploy eduforge-ingestion \
  --image="${REGISTRY}/ingestion:${TAG}" \
  --region=$REGION \
  --service-account=$INGESTION_SA \
  --no-allow-unauthenticated \
  --memory=2Gi \
  --cpu=2 \
  --min-instances=0 \
  --max-instances=5 \
  --timeout=3600 \
  --set-env-vars="GOOGLE_CLOUD_PROJECT=${PROJECT_ID}" \
  --set-secrets="GEMINI_API_KEY=GEMINI_API_KEY:latest,DOCUMENT_AI_PROCESSOR_ID=DOCUMENT_AI_PROCESSOR_ID:latest,DOCUMENT_AI_LOCATION=DOCU
  --project=$PROJECT_ID

INGESTION_URL=$(gcloud run services describe eduforge-ingestion \
  --region=$REGION \
  --format="value(status.url)" \
  --project=$PROJECT_ID)
echo "✅ Ingestion Service: $INGESTION_URL"

# ── UPDATE PUB/SUB PUSH ENDPOINTS ─────────────────────────────────────
echo "--- Updating Pub/Sub push endpoints ---"
gcloud pubsub subscriptions modify-push-config lesson-ingestion-sub \
  --push-endpoint="${INGESTION_URL}/trigger" \
  --project=$PROJECT_ID

gcloud pubsub subscriptions modify-push-config lesson-complete-sub \
  --push-endpoint="${API_URL}/internal/lesson-complete" \
  --project=$PROJECT_ID

echo "✅ Pub/Sub endpoints updated"

# ── DEPLOY FRONTEND ───────────────────────────────────────────────────
echo "--- Deploying Frontend to Firebase Hosting ---"
cd apps/web

# Build Next.js with environment variables
NEXT_PUBLIC_API_URL=$API_URL \
NEXT_PUBLIC_BKT_URL=$BKT_URL \
NEXT_PUBLIC_GENUI_URL=$GENUI_URL \
npm run build

firebase deploy --only hosting --project=$PROJECT_ID

FRONTEND_URL="https://${PROJECT_ID}.web.app"
echo "✅ Frontend: $FRONTEND_URL"

cd ../..

# ── DEPLOY FIRESTORE RULES & INDEXES ──────────────────────────────────
echo "--- Deploying Firestore security rules ---"
firebase deploy --only firestore --project=$PROJECT_ID
echo "✅ Firestore rules deployed"

echo ""
echo "╔═══════════════════════════════════════════════════════╗"
echo "║           EduForge Deployment Complete! 🎉            ║"
echo "╠═══════════════════════════════════════════════════════╣"
echo "║  Frontend:   $FRONTEND_URL"
echo "║  API:        $API_URL"
echo "║  BKT:        $BKT_URL"
echo "║  GenUI:      $GENUI_URL"
echo "║  Ingestion:  $INGESTION_URL"
echo "╚═══════════════════════════════════════════════════════╝"
```

17.3 Local Development Setup

```bash
#!/bin/bash
# scripts/dev/start_local.sh
# Starts all services locally with docker-compose

set -e

echo "Starting EduForge local development environment..."

# Copy environment template if .env.local doesn't exist
if [ ! -f .env.local ]; then
  cp .env.example .env.local
  echo "⚠️  Created .env.local from template. Fill in your API keys before proceeding."
  exit 1
fi

# Start all services
docker-compose up --build

echo "✅ All services started"
echo "  Frontend: http://localhost:3000"
echo "  API:      http://localhost:8000"
echo "  BKT:      http://localhost:8001"
echo "  GenUI:    http://localhost:8002"
```

```yaml
# docker-compose.yml
version: '3.8'
services:
  web:
    build: ./apps/web
    ports: ["3000:3000"]
    environment:
      - NEXT_PUBLIC_API_URL=http://localhost:8000
      - NEXT_PUBLIC_BKT_URL=http://localhost:8001
      - NEXT_PUBLIC_GENUI_URL=http://localhost:8002
    env_file: .env.local
    depends_on: [api, bkt-service, genui-service]
    volumes:
      - ./apps/web/src:/app/src

  api:
    build: ./apps/api
    ports: ["8000:8080"]
    environment:
      - GOOGLE_CLOUD_PROJECT=${PROJECT_ID}
      - BKT_SERVICE_URL=http://bkt-service:8080
    env_file: .env.local

  bkt-service:
    build: ./apps/bkt-service
    ports: ["8001:8080"]
    environment:
      - GOOGLE_CLOUD_PROJECT=${PROJECT_ID}
    env_file: .env.local

  genui-service:
    build: ./apps/genui-service
    ports: ["8002:8080"]
    env_file: .env.local

  ingestion:
    build: ./apps/ingestion
    ports: ["8003:8080"]
    environment:
      - GOOGLE_CLOUD_PROJECT=${PROJECT_ID}
    env_file: .env.local
```

17.4 Environment Variables Template

```
# .env.example — copy to .env.local and fill in values

# GCP
PROJECT_ID=your-gcp-project-id
GOOGLE_CLOUD_PROJECT=your-gcp-project-id
GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account.json

# Firebase (get from Firebase Console → Project Settings)
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=your-project.firebaseapp.com
NEXT_PUBLIC_FIREBASE_PROJECT_ID=your-project-id
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=your-project.appspot.com
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=
FIREBASE_SERVICE_ACCOUNT_JSON={"type":"service_account",...}

# AI APIs
CLAUDE_API_KEY=sk-ant-...
GEMINI_API_KEY=AIza...

# Document AI
DOCUMENT_AI_PROCESSOR_ID=your-processor-id
DOCUMENT_AI_LOCATION=us

# Service URLs (local dev)
NEXT_PUBLIC_API_URL=http://localhost:8000
BKT_SERVICE_URL=http://localhost:8001
GENUI_SERVICE_URL=http://localhost:8002

# Storage
LESSON_UPLOADS_BUCKET=your-project-lesson-uploads
LESSON_ASSETS_BUCKET=your-project-lesson-assets

# App
NEXTAUTH_SECRET=your-random-secret
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

17.5 Seed Demo Data for Competition

```
#!/usr/bin/env python3
# scripts/seed/seed_demo_lesson.py
"""
Creates a realistic demo lesson (Newton's Laws of Motion — Physics Grade 10)
with pre-populated subtopics, MCQs, and simulated student BKT states
for the competition demo.
"""

import asyncio
import json
from google.cloud import firestore
from datetime import datetime

db = firestore.AsyncClient()

DEMO_LESSON = {
    "id": "demo_newtons_laws",
    "title": "Newton's Laws of Motion",
    "subject": "Physics",
    "teacherId": "demo_teacher_1",
    "status": "published",
    "createdAt": datetime.now(),
    "publishedAt": datetime.now(),
    "ingestion": {
        "step": "complete",
        "progress": 100,
        "message": "Lesson ready!",
        "subtopicsFound": 4,
        "mcqsGenerated": 60,
    }
}

DEMO_SUBTOPICS = [
    {
        "id": "subtopic_1",
        "title": "Inertia and Newton's First Law",
        "description": "Objects at rest stay at rest; objects in motion stay in motion unless acted upon by a force.",
        "order": 1,
        "difficulty": "foundational",
        "slideNumbers": [1, 2, 3],
        "keyConcepts": ["inertia", "net force", "balanced forces"],
        "learningObjectives": ["Define inertia", "Identify balanced and unbalanced forces", "Apply First Law to real scenarios"],
        "prerequisiteSubtopicIds": [],
    },
    {
        "id": "subtopic_2",
        "title": "Force, Mass, and Acceleration: Newton's Second Law",
        "description": "The acceleration of an object depends on the net force and its mass: F = ma.",
        "order": 2,
        "difficulty": "intermediate",
        "slideNumbers": [4, 5, 6, 7],
        "keyConcepts": ["net force", "mass", "acceleration", "F=ma"],
        "learningObjectives": ["Apply F=ma formula", "Calculate acceleration given force and mass", "Understand inverse relationship of
        "prerequisiteSubtopicIds": ["subtopic_1"],
    },
    {
        "id": "subtopic_3",
        "title": "Action and Reaction: Newton's Third Law",
        "description": "For every action there is an equal and opposite reaction force.",
        "order": 3,
        "difficulty": "intermediate",
        "slideNumbers": [8, 9, 10],
        "keyConcepts": ["action-reaction pairs", "force pairs", "equal and opposite"],
        "learningObjectives": ["Identify action-reaction pairs", "Explain why reaction forces don't cancel", "Apply Third Law to everyda
        "prerequisiteSubtopicIds": ["subtopic_1"],
    },
    {
        "id": "subtopic_4",
        "title": "Applying All Three Laws: Problem Solving",
        "description": "Integrating all three Newtonian laws to solve multi-step physics problems.",
        "order": 4,
        "difficulty": "advanced",
        "slideNumbers": [11, 12, 13, 14, 15],
        "keyConcepts": ["free body diagrams", "problem solving framework", "vector forces"],
        "learningObjectives": ["Draw free body diagrams", "Identify applicable laws", "Solve multi-step force problems"],
        "prerequisiteSubtopicIds": ["subtopic_2", "subtopic_3"],
```

```python
    },
]

# Two demo students with different BKT states to illustrate side-by-side demo
DEMO_STUDENTS = [
    {
        "id": "demo_student_alice",
        "name": "Alice",
        # Alice: has completed subtopics 1 & 2, currently on 3
        "bkt_states": {
            "subtopic_1": {"pMastery": 0.95, "mastered": True, "attempts": 8},
            "subtopic_2": {"pMastery": 0.87, "mastered": True, "attempts": 12},
            "subtopic_3": {"pMastery": 0.45, "mastered": False, "attempts": 4},
            "subtopic_4": {"pMastery": 0.2, "mastered": False, "attempts": 0},
        }
    },
    {
        "id": "demo_student_bob",
        "name": "Bob",
        # Bob: stuck on subtopic 1, hasn't unlocked 2
        "bkt_states": {
            "subtopic_1": {"pMastery": 0.28, "mastered": False, "attempts": 6},
            "subtopic_2": {"pMastery": 0.2, "mastered": False, "attempts": 0},
            "subtopic_3": {"pMastery": 0.2, "mastered": False, "attempts": 0},
            "subtopic_4": {"pMastery": 0.2, "mastered": False, "attempts": 0},
        }
    },
]

async def seed():
    print("Seeding demo data...")

    # Create lesson
    await db.collection('lessons').document(DEMO_LESSON['id']).set(DEMO_LESSON)

    # Create subtopics
    for st in DEMO_SUBTOPICS:
        await db.collection('lessons').document(DEMO_LESSON['id'])\
                .collection('subtopics').document(st['id']).set(st)

    # Create demo user profiles
    for student in DEMO_STUDENTS:
        await db.collection('users').document(student['id']).set({
            "uid": student['id'],
            "displayName": student['name'],
            "role": "student",
            "email": f"{student['name'].lower()}@demo.eduforge.app",
        })

        # Set BKT states
        for subtopic_id, state in student['bkt_states'].items():
            concept_id = f"{subtopic_id}_inertia"  # Using first concept of each subtopic
            await db.collection('bkt_states').document(student['id'])\
                    .collection('concepts').document(concept_id).set({
                        "studentId": student['id'],
                        "lessonId": DEMO_LESSON['id'],
                        "subtopicId": subtopic_id,
                        "conceptId": concept_id,
                        "lastUpdated": datetime.now(),
                        **state
                    })

        # Create enrollment
        enrollment_id = f"{student['id']}_{DEMO_LESSON['id']}"
        await db.collection('enrollments').document(enrollment_id).set({
            "studentId": student['id'],
            "lessonId": DEMO_LESSON['id'],
            "enrolledAt": datetime.now(),
            "status": "active",
        })

    print("✅ Demo data seeded successfully")
    print(f"   Lesson: {DEMO_LESSON['title']}")
    print(f"   Subtopics: {len(DEMO_SUBTOPICS)}")
    print(f"   Students: {[s['name'] for s in DEMO_STUDENTS]}")

if __name__ == '__main__':
    asyncio.run(seed())
```

18. Day-by-Day Build Timeline

Day 1 — Foundation & Infrastructure

- [ ] Run all GCP setup scripts (sections 3.1–3.10)
- [ ] Initialize Firebase project + link to GCP
- [ ] Set up monorepo structure with `pnpm workspaces`
- [ ] Create `packages/shared` types
- [ ] Deploy Firebase Auth + basic Next.js login/register pages
- [ ] Scaffold all 4 backend services with health endpoints
- [ ] Push Docker images, deploy to Cloud Run (hello world)
- [ ] Verify Pub/Sub topics and subscriptions are connected

End of Day 1 Goal: All GCP services running, auth working, CI builds green.

Day 2 — Ingestion Pipeline

- [ ] PPTX extractor (`python-pptx` + image upload to GCS)
- [ ] Cloud Document AI integration (Layout Parser)
- [ ] Gemini topic hierarchy generator
- [ ] Gemini MCQ generator (parallel per subtopic)
- [ ] BKT parameter seeder
- [ ] Firestore write pipeline (lesson → subtopics → MCQs → BKT params)
- [ ] Teacher upload UI + signed URL upload + ingestion status real-time updates
- [ ] Test with a real PPTX file end-to-end

End of Day 2 Goal: Upload a PPTX → see topics appear in Firestore within 5 minutes.

Day 3 — BKT Engine + Student Routing

- [ ] BKT engine (core update equations, tested)
- [ ] Scaffold resolver (5 levels → 5 component catalogs)
- [ ] BKT service endpoints (`/update`, `/state`, `/scaffold`)
- [ ] Student topic index (with BKT-driven lock/unlock)
- [ ] Adaptive MCQ component (3 tiers, feedback, misconception alerts)
- [ ] Mastery HUD (D3 radar chart)
- [ ] BKT real-time Firestore listener (`useBKTState` hook)
- [ ] Write BKT unit tests (all passing)

End of Day 3 Goal: Student can answer MCQs and see mastery update in real-time.

Day 4 — GenUI Layer + AI Tutor

- [ ] OpenUI component library definitions (all 8 components)
- [ ] `StepByStep`, `HintCard`, `FormulaCard`, `AnalogyCard` React implementations
- [ ] `ConceptDiagram`, `PracticeExercise`, `ProofWalkthrough`, `ExpertSummary` implementations
- [ ] GenUI prompt builder (BKT state → system prompt with component catalog)
- [ ] GenUI streaming service (Claude API → SSE stream)
- [ ] Next.js streaming route (`/api/genui`)
- [ ] AI Tutor Chat (BKT-gated system prompt, streaming responses)
- [ ] Full student learning page layout (topic index + 4-panel GenUI area)

End of Day 4 Goal: Student gets a GenUI visualization that visibly adapts to their mastery level.

Day 5 — Teacher Dashboard + Analytics

- [ ] Teacher lesson management dashboard
- [ ] Class mastery heatmap (Firestore aggregation)
- [ ] Student progress tables
- [ ] Ingestion status notifications (bell icon, real-time)
- [ ] Concept graph editor (React Flow, teacher edits auto-generated graph)
- [ ] MCQ bank preview/edit (teacher can regenerate individual questions)
- [ ] Seed demo data for competition (`seed_demo_lesson.py`)

End of Day 5 Goal: Full teacher → publish → student learn → teacher sees analytics loop working.

Day 6 — Polish, Testing, Deployment

- [ ] Run full test suite (`run_all_tests.sh`)
- [ ] UI polish pass (loading states, empty states, error boundaries)
- [ ] Deploy all services to production (`deploy_all.sh`)
- [ ] Test competition demo flow end-to-end on prod
- [ ] Record demo video backup
- [ ] Final documentation pass

---

19. Competition Demo Script

The Setup

- Pre-create teacher account: `teacher@demo.eduforge.app`
- Pre-create two student accounts: `alice@demo.eduforge.app`, `bob@demo.eduforge.app`
- Pre-seed the demo lesson (Newton's Laws) using `seed_demo_lesson.py`
- Open 3 browser windows side by side: teacher, Alice, Bob

Demo Flow (8 minutes)

Minute 1-2: Teacher Portal

1. Log in as teacher → show class dashboard
2. Upload a real PPTX (Newton's Laws slides)
3. Watch the real-time ingestion status update (queued → extracting → generating topics → complete)
4. Show the auto-generated topic index: 4 subtopics, MCQ counts per subtopic
5. Show the concept dependency graph editor

Minute 3-4: The Side-by-Side Reveal

1. Open Alice and Bob on same subtopic: "Inertia and Newton's First Law"
2. **Key demo moment**: Alice (P=0.45, scaffold level 2) sees `ConceptDiagram` + `PracticeExercise`
3. Bob (P=0.28, scaffold level 1) sees `StepByStep` with numbered steps + `HintCard`
4. Say: "Same lesson. Same teacher. Same subtopic. Two completely different interfaces — each generated in real-time by Claude, constrained by their personal knowledge state."

Minute 5-6: Live BKT Update

1. Have Bob answer an MCQ correctly → mastery bar ticks up live
2. Have Bob answer 2 more correctly → watch scaffold level change
3. Refresh Bob's GenUI panel → new component appears (less scaffolding)
4. Show mastery HUD radar chart updating in real-time

Minute 7: AI Tutor Differentiation

1. Both Alice and Bob ask the same question: "Can you explain why a ball keeps rolling?"
2. Alice (scaffold 2): AI explains with moderate detail, asks a follow-up question
3. Bob (scaffold 0): AI gives a full step-by-step analogy, much more detailed

Minute 8: Teacher Analytics

1. Switch to teacher dashboard → class heatmap shows Bob is struggling on Inertia
2. Show "students at risk" alert
3. Close with: "This is the first system where the knowledge state — not the question — drives the entire user interface."

Appendix A: Key Research References

1. **Leviathan, Y. & Valevski, D. (2025).** "Generative UI: LLMs are Effective UI Generators." Google Research. https://generativeui.github.io/
2. **Badrinath, A., Wang, F., & Pardos, Z.A. (2021).** "pyBKT: An Accessible Python Library of Bayesian Knowledge Tracing Models." EDM 2021.
3. **Corbett, A.T. & Anderson, J.R. (1994).** "Knowledge tracing: Modeling the acquisition of procedural knowledge." *User Modeling and User-Adapted Interaction*, 4(4).
4. **Thesysdev (2025).** "OpenUI: The Open Standard for Generative UI." https://openui.com
5. **CopilotKit (2025).** "AG-UI Protocol: Generative UI Patterns." https://copilotkit.ai/generative-ui
6. **Google Cloud (2025).** "Document AI Layout Parser — PPTX Support GA." https://cloud.google.com/document-ai/docs/release-notes

---

20. pyBKT + GenUI Deep Integration Guide

> **This section provides the exact code, AI prompts, and integration patterns for wiring pyBKT's Roster API into the GenUI pipeline. This is the core neuro-symbolic fusion that makes EduForge unique.**

20.1 pyBKT Library — Proper Integration (v1.4.2)

The existing BKT engine in Section 7 uses a manual Bayesian update. **Replace the manual update with pyBKT's Roster API** for production — it handles state management, mastery detection, and supports batch parameter fitting from historical data.

Why Roster over raw BKT equations:

- pyBKT's `Roster` class stores only the current latent/observable state per student — memory efficient
- `Roster.update_state()` handles the full HMM forward pass internally
- `Roster.get_mastery_prob()` returns calibrated P(mastery) aligned with fitted parameters
- `StateType.MASTERED` / `StateType.UNMASTERED` provides clean mastery classification
- Supports adding/removing students dynamically — perfect for a live classroom

Installation in Docker (Cloud Run)

```
# apps/bkt-service/Dockerfile
FROM python:3.11-slim

# Install C++ compiler for fast pyBKT inferencing (150-600x speedup)
RUN apt-get update && apt-get install -y gcc g++ && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

```
# apps/bkt-service/requirements.txt
pyBKT==1.4.2
fastapi==0.115.6
uvicorn[standard]==0.34.0
google-cloud-firestore==2.20.0
numpy==1.26.4
pandas==2.2.3
pydantic==2.10.3
```

20.1.1 pyBKT Roster-Based Engine (Replace Section 7.2)

```python
# apps/bkt-service/src/bkt/engine.py
"""
Production BKT engine using pyBKT's Roster API.
Replaces the manual Bayesian update with pyBKT's HMM forward pass.

Key difference from Section 7.2:
- Uses pyBKT's fitted parameters (EM-optimized) instead of hand-tuned defaults
- Roster manages per-student state efficiently
- Supports batch parameter re-fitting as response data accumulates
"""

from pyBKT.models import Model, Roster, StateType
import numpy as np
import pandas as pd
import json
import os
import logging
from dataclasses import dataclass, field
from typing import Optional
from google.cloud import firestore

logger = logging.getLogger(__name__)

@dataclass
class BKTParams:
    concept_id: str
    p_initial: float = 0.2
    p_learn: float = 0.2
    p_slip: float = 0.1
    p_guess: float = 0.25

@dataclass
class StudentConceptState:
    student_id: str
    concept_id: str
    subtopic_id: str
    p_mastery: float = 0.2
    attempts: int = 0
    correct_streak: int = 0
    mastered: bool = False
    consecutive_wrong: int = 0
    response_history: list = field(default_factory=list)


class PyBKTEngine:
    """
    Production BKT engine using pyBKT v1.4.2 Roster API.

    Architecture:
    1. On lesson publish: fit a pyBKT Model per skill from seed data
    2. Per student-concept pair: create a Roster entry
    3. On each MCQ answer: Roster.update_state() → get_mastery_prob()
    4. Periodically: re-fit Model from accumulated response data (improves accuracy)
    """

    MASTERY_THRESHOLD = 0.95

    def __init__(self):
        self._models: dict[str, Model] = {}       # concept_id → fitted Model
        self._rosters: dict[str, Roster] = {}      # concept_id → Roster
        self.db = firestore.AsyncClient()

    async def initialize_concept(self, concept_id: str, params: BKTParams):
        """
        Initialize a pyBKT Model for a concept with seed parameters.
        Called once per concept when a lesson is published.
        """
        model = Model(seed=42, num_fits=1)

        # Set initial parameters via model.coef_ (parameter fixing)
        # This seeds the model with our Gemini-estimated params from ingestion
        model.coef_ = {
            concept_id: {
                'prior': params.p_initial,
                'learns': np.array([params.p_learn]),
                'guesses': np.array([params.p_guess]),
```

```
                'slips': np.array([params.p_slip]),
            }
        }

        # Generate minimal synthetic training data to initialize the model
        # pyBKT requires a fit() call before Roster can be created
        seed_df = self._generate_seed_data(concept_id, params)
        model.fit(data=seed_df, skills=concept_id, fixed=True)

        self._models[concept_id] = model
        logger.info(f"Initialized pyBKT model for concept: {concept_id}")
        logger.info(f"  Params: P(L0)={params.p_initial}, P(T)={params.p_learn}, "
                    f"P(S)={params.p_slip}, P(G)={params.p_guess}")

    def _generate_seed_data(self, concept_id: str, params: BKTParams) -> pd.DataFrame:
        """
        Generate minimal synthetic data for pyBKT model initialization.
        pyBKT requires at least some data to run fit() and create a Roster.
        """
        np.random.seed(42)
        n_students = 20
        n_questions = 10

        rows = []
        for student_idx in range(n_students):
            mastered = False
            p_know = params.p_initial

            for q in range(n_questions):
                if mastered:
                    correct = 1 if np.random.random() > params.p_slip else 0
                else:
                    correct = 1 if np.random.random() < params.p_guess else 0
                    if np.random.random() < params.p_learn:
                        mastered = True

                rows.append({
                    'order_id': student_idx * n_questions + q,
                    'skill_name': concept_id,
                    'correct': correct,
                    'user_id': f'seed_student_{student_idx}',
                })

        return pd.DataFrame(rows)

    async def get_or_create_roster(self, concept_id: str, student_id: str) -> Roster:
        """Get existing Roster or create one for this concept."""
        if concept_id not in self._rosters:
            if concept_id not in self._models:
                raise ValueError(f"Model not initialized for concept: {concept_id}")

            self._rosters[concept_id] = Roster(
                students=[student_id],
                skills=concept_id,
                model=self._models[concept_id]
            )
        else:
            roster = self._rosters[concept_id]
            # Add student if not already in roster
            try:
                roster.get_mastery_prob(concept_id, student_id)
            except (KeyError, Exception):
                roster.add_student(concept_id, student_id)

        return self._rosters[concept_id]

    async def update(
        self,
        student_id: str,
        concept_id: str,
        subtopic_id: str,
        is_correct: bool,
        current_state: Optional[StudentConceptState] = None,
    ) -> StudentConceptState:
        """
        Update BKT state after a student answers a question.
        Uses pyBKT Roster.update_state() for the actual HMM update.
        """
        roster = await self.get_or_create_roster(concept_id, student_id)
```

```python
        # pyBKT expects: 1 = correct, 0 = incorrect
        response = np.array([1 if is_correct else 0])

        # Run the pyBKT HMM forward update
        new_state_obj = roster.update_state(concept_id, student_id, response)

        # Extract P(mastery) from pyBKT
        p_mastery = float(roster.get_mastery_prob(concept_id, student_id))
        state_type = roster.get_state_type(concept_id, student_id)
        is_mastered = state_type == StateType.MASTERED

        # Clamp to valid range
        p_mastery = max(0.001, min(0.999, p_mastery))

        # If pyBKT says mastered but below our threshold, trust the threshold
        if p_mastery >= self.MASTERY_THRESHOLD:
            is_mastered = True

        # Build updated state object with tracking metadata
        prev_state = current_state or StudentConceptState(
            student_id=student_id,
            concept_id=concept_id,
            subtopic_id=subtopic_id,
        )

        new_streak = prev_state.correct_streak + 1 if is_correct else 0
        new_consecutive_wrong = 0 if is_correct else prev_state.consecutive_wrong + 1

        response_entry = {
            'is_correct': is_correct,
            'p_mastery_before': prev_state.p_mastery,
            'p_mastery_after': p_mastery,
        }

        updated = StudentConceptState(
            student_id=student_id,
            concept_id=concept_id,
            subtopic_id=subtopic_id,
            p_mastery=p_mastery,
            attempts=prev_state.attempts + 1,
            correct_streak=new_streak,
            mastered=is_mastered,
            consecutive_wrong=new_consecutive_wrong,
            response_history=(prev_state.response_history + [response_entry])[-20:],
        )

        logger.info(
            f"BKT update: student={student_id}, concept={concept_id}, "
            f"correct={is_correct}, P(mastery): {prev_state.p_mastery:.3f} -> {p_mastery:.3f}, "
            f"mastered={is_mastered}"
        )

        return updated

    def predict_next_correct(self, concept_id: str, student_id: str) -> float:
        """Predict probability of correct answer on next question."""
        if concept_id in self._rosters:
            roster = self._rosters[concept_id]
            try:
                probs = roster.get_correct_probs(concept_id)
                if student_id in probs:
                    return float(probs[student_id])
            except Exception:
                pass
        return 0.5  # default if not in roster

    async def refit_from_data(self, concept_id: str, response_data: pd.DataFrame):
        """
        Re-fit pyBKT parameters from accumulated real student data.
        Call this periodically (e.g., nightly cron) to improve accuracy.

        response_data columns: order_id, skill_name, correct, user_id
        """
        if len(response_data) < 50:
            logger.info(f"Skipping refit for {concept_id}: insufficient data ({len(response_data)} responses)")
            return

        model = Model(seed=42, num_fits=5)
        model.fit(
            data=response_data,
```

```
            skills=concept_id,
            defaults={
                'order_id': 'order_id',
                'skill_name': 'skill_name',
                'correct': 'correct',
                'user_id': 'user_id',
            }
        )

        params = model.params()
        logger.info(f"Re-fitted BKT params for {concept_id}: {params}")

        self._models[concept_id] = model

        # Recreate rosters with updated model
        if concept_id in self._rosters:
            del self._rosters[concept_id]

        # Persist new params to Firestore
        # (students will get updated model on next interaction)
```

20.1.2 Batch Parameter Fitting from Historical Data

```python
# apps/bkt-service/src/bkt/param_fitter.py
"""
Scheduled job: Re-fit BKT parameters from accumulated student response data.
Run nightly via Cloud Scheduler → Pub/Sub → Cloud Run trigger.

After 50+ responses per concept, pyBKT can fit better params than seed defaults.
This is the adaptive learning loop: student performance → better model → better scaffolding.
"""

from pyBKT.models import Model
import pandas as pd
from google.cloud import firestore
import logging

logger = logging.getLogger(__name__)

class BKTParamFitter:
    def __init__(self, project_id: str):
        self.db = firestore.AsyncClient(project=project_id)

    async def fit_all_concepts(self, lesson_id: str):
        """Fit BKT params for all concepts in a lesson from real response data."""

        # Query all responses for this lesson
        responses_ref = self.db.collection('responses')
        query = responses_ref.where('lessonId', '==', lesson_id)
        docs = [doc async for doc in query.stream()]

        if len(docs) < 50:
            logger.info(f"Lesson {lesson_id}: only {len(docs)} responses, skipping refit")
            return

        # Convert to DataFrame
        rows = []
        for doc in docs:
            d = doc.to_dict()
            rows.append({
                'order_id': doc.id,
                'skill_name': d['conceptId'],
                'correct': 1 if d['isCorrect'] else 0,
                'user_id': d['studentId'],
            })

        df = pd.DataFrame(rows)

        # Fit per-concept
        concepts = df['skill_name'].unique()
        for concept_id in concepts:
            concept_df = df[df['skill_name'] == concept_id]
            if len(concept_df) < 30:
                continue

            try:
                model = Model(seed=42, num_fits=5)
                model.fit(data=concept_df, skills=concept_id, defaults={
                    'order_id': 'order_id',
                    'skill_name': 'skill_name',
                    'correct': 'correct',
                    'user_id': 'user_id',
                })

                params = model.params()
                fitted = params[concept_id]

                # Persist to Firestore
                await self.db.collection('lessons').document(lesson_id)\
                    .collection('bkt_params').document(concept_id).update({
                        'pInitial': float(fitted['prior']),
                        'pLearn': float(fitted['learns'][0]),
                        'pSlip': float(fitted['slips'][0]),
                        'pGuess': float(fitted['guesses'][0]),
                        'fittedFromResponses': len(concept_df),
                        'lastFittedAt': firestore.SERVER_TIMESTAMP,
                    })

                logger.info(
```

```
                f"Re-fitted {concept_id}: P(L0)={fitted['prior']:.3f}, "
                f"P(T)={fitted['learns'][0]:.3f}, P(S)={fitted['slips'][0]:.3f}, "
                f"P(G)={fitted['guesses'][0]:.3f} (from {len(concept_df)} responses)"
            )

        except Exception as e:
            logger.error(f"Failed to fit {concept_id}: {e}")

    async def evaluate_model(self, lesson_id: str, concept_id: str) -> dict:
        """
        Cross-validate the BKT model for a concept.
        Returns accuracy, AUC, and RMSE metrics.
        """
        responses_ref = self.db.collection('responses')
        query = responses_ref.where('lessonId', '==', lesson_id).where('conceptId', '==', concept_id)
        docs = [doc async for doc in query.stream()]

        rows = [{
            'order_id': doc.id,
            'skill_name': concept_id,
            'correct': 1 if doc.to_dict()['isCorrect'] else 0,
            'user_id': doc.to_dict()['studentId'],
        } for doc in docs]

        df = pd.DataFrame(rows)

        model = Model(seed=42, num_fits=5)

        # 5-fold cross-validation
        cv_rmse = model.crossvalidate(data=df, skills=concept_id, folds=5, metric='rmse', defaults={
            'order_id': 'order_id', 'skill_name': 'skill_name',
            'correct': 'correct', 'user_id': 'user_id',
        })

        cv_auc = model.crossvalidate(data=df, skills=concept_id, folds=5, metric='auc', defaults={
            'order_id': 'order_id', 'skill_name': 'skill_name',
            'correct': 'correct', 'user_id': 'user_id',
        })

        cv_acc = model.crossvalidate(data=df, skills=concept_id, folds=5, metric='accuracy', defaults={
            'order_id': 'order_id', 'skill_name': 'skill_name',
            'correct': 'correct', 'user_id': 'user_id',
        })

        return {
            'concept_id': concept_id,
            'rmse': float(cv_rmse[concept_id]),
            'auc': float(cv_auc[concept_id]),
            'accuracy': float(cv_acc[concept_id]),
            'n_responses': len(df),
        }
```

20.2 GenUI — AI Prompts for BKT-Constrained Component Generation

The GenUI system prompt is the most critical piece — it must enforce that the LLM only uses components from the BKT-gated catalog. Below are the **exact production prompts** for each scaffold level.

20.2.1 System Prompt Builder (Complete Implementation)

```
# apps/genui-service/src/genui/prompt_builder.py
"""
Builds the system prompt that goes to Claude for GenUI generation.
The BKT state determines which components the LLM is ALLOWED to emit.

This is the neuro-symbolic constraint:
  Symbolic (BKT posterior) → constrains → Neural (LLM generation)
"""

from typing import Optional

class GenUIPromptBuilder:

    # Component schema definitions — injected into the LLM system prompt
    # so Claude knows the exact props for each component
    COMPONENT_SCHEMAS = {
        'StepByStep': '''{
  "component": "StepByStep",
  "props": {
    "concept": "string — the concept being explained",
    "steps": [
      {
        "number": 1,
        "title": "string — step title",
        "explanation": "string — detailed explanation",
        "example": "string? — concrete example"
      }
    ],
    "summary": "string? — brief recap"
  }
}''',
        'HintCard': '''{
  "component": "HintCard",
  "props": {
    "hint_level": "gentle | moderate | direct",
    "hint_text": "string — the hint content",
    "follow_up_question": "string? — Socratic follow-up"
  }
}''',
        'FormulaCard': '''{
  "component": "FormulaCard",
  "props": {
    "formula": "string — formula in LaTeX or plain text",
    "variables": [
      {"symbol": "F", "name": "Force", "unit": "Newtons (N)"}
    ],
    "example": {
      "values": {"F": "10 N", "m": "2 kg"},
      "result": "a = 5 m/s²"
    }
  }
}''',
        'ConceptDiagram': '''{
  "component": "ConceptDiagram",
  "props": {
    "title": "string",
    "diagram_type": "process_flow | comparison | hierarchy | cycle | relationship",
    "elements": [
      {"id": "e1", "label": "string", "description": "string?", "connects_to": ["e2"]}
    ],
    "annotations": ["string — key observation"]
  }
}''',
        'AnalogyCard': '''{
  "component": "AnalogyCard",
  "props": {
    "abstract_concept": "string — the concept being analogized",
    "real_world_analogy": "string — the relatable example",
    "how_they_match": [
      {"concept_aspect": "Inertia", "analogy_aspect": "A ball sitting on a table stays put"}
    ],
    "limitation": "string? — where the analogy breaks down"
  }
}''',
        'PracticeExercise': '''{
  "component": "PracticeExercise",
```

```
  "props": {
    "problem": "string — the exercise problem statement",
    "hints": ["string — progressive hints"],
    "worked_solution": "string — complete solution",
    "key_insight": "string — the takeaway"
  }
}''',
        'ProofWalkthrough': '''{
  "component": "ProofWalkthrough",
  "props": {
    "theorem": "string — the theorem being proved",
    "proof_steps": [
      {"step": 1, "statement": "string", "justification": "string"}
    ],
    "conclusion": "string"
  }
}''',
        'ExpertSummary': '''{
  "component": "ExpertSummary",
  "props": {
    "key_ideas": ["string — core ideas"],
    "common_pitfalls": ["string — common mistakes to avoid"],
    "advanced_connections": ["string — links to broader concepts"],
    "challenge_question": "string — pushes beyond the lesson"
  }
}''',
    }

    # Pedagogical behavior rules per scaffold level
    PEDAGOGICAL_RULES = {
        0: """SCAFFOLD LEVEL 0 — NOVICE (P(mastery) < 0.20)
STUDENT IS A COMPLETE BEGINNER. They have no prior understanding of this concept.

REQUIRED BEHAVIOR:
- Use StepByStep as the PRIMARY component — break everything into small, numbered steps (max 8 steps)
- ALWAYS include a FormulaCard if the concept involves any formula or equation
- Use AnalogyCard to connect abstract ideas to everyday experiences the student already understands
- Use HintCard with hint_level="gentle" — give full, explicit explanations
- Language: simple, short sentences. NO jargon. Define every technical term inline.
- Tone: warm, encouraging, patient. Use phrases like "Let's start with the basics..." and "Think of it like this..."
- NEVER assume prior knowledge. NEVER skip steps. NEVER use "obviously" or "simply".
- Each step explanation should be 2-3 sentences maximum.
- Include a concrete, real-world example for EVERY step.""",

        1: """SCAFFOLD LEVEL 1 — DEVELOPING (P(mastery) 0.20–0.40)
STUDENT HAS SOME AWARENESS but doesn't reliably understand the concept.

REQUIRED BEHAVIOR:
- Use StepByStep but can group simpler sub-steps together (max 6 steps)
- Include ConceptDiagram with partial labels — let the student make some connections
- FormulaCard should show the formula with a worked example
- HintCard with hint_level="moderate" — explain clearly but don't over-scaffold
- Language: accessible but can introduce technical terms WITH definitions
- Tone: supportive. "You've seen some of this before — let's build on that..."
- Focus on UNDERSTANDING, not just recall. Ask "why" not just "what"."""

        2: """SCAFFOLD LEVEL 2 — APPROACHING (P(mastery) 0.40–0.60)
STUDENT HAS PARTIAL UNDERSTANDING. They get the basics but struggle with application.

REQUIRED BEHAVIOR:
- Use ConceptDiagram as the PRIMARY component — fully labeled, showing relationships
- Include PracticeExercise with progressive hints (3 hints: gentle → moderate → direct)
- FormulaCard for reference only, not as the main explanation
- HintCard only if the student specifically asks or gets stuck — use hint_level="moderate"
- Language: standard academic. Technical terms are fine without definitions.
- Tone: collaborative. "Let's apply what you know..."
- Focus on APPLICATION and CONNECTIONS between ideas.
- Challenge: include one PracticeExercise that requires combining 2+ concepts.""",

        3: """SCAFFOLD LEVEL 3 — PROFICIENT (P(mastery) 0.60–0.80)
STUDENT HAS GOOD UNDERSTANDING. They can apply the concept reliably.

REQUIRED BEHAVIOR:
- Minimal scaffolding. NO step-by-step walkthroughs.
- Use ConceptDiagram showing advanced relationships and edge cases
- PracticeExercise with harder problems — multi-step, require synthesis
- ProofWalkthrough if the concept has a formal proof or derivation
- Language: precise, academic. Assume familiarity with core terminology.
- Tone: collegial. "Consider this edge case..." or "How would this change if..."
- Focus on EDGE CASES, EXCEPTIONS, and SYNTHESIS across concepts.
```

```
- NEVER re-explain basics. If you catch yourself explaining foundational ideas, stop.""",

        4: """SCAFFOLD LEVEL 4 — MASTERED (P(mastery) > 0.80)
STUDENT IS AN EXPERT ON THIS CONCEPT. Challenge them.

REQUIRED BEHAVIOR:
- Use ONLY: ConceptDiagram, ExpertSummary, ProofWalkthrough, PracticeExercise
- ExpertSummary: dense, expert-level recap with common pitfalls and advanced connections
- PracticeExercise: competition-level problems that require creative application
- ProofWalkthrough: formal derivations with rigorous justification
- Use SOCRATIC METHOD: ask leading questions instead of giving answers directly
- Language: expert-level. Assume deep familiarity.
- Tone: Socratic challenger. "What happens if we remove this constraint?" or "Can you generalize this?"
- NEVER give direct answers. Guide through questions.
- Include a challenge_question in ExpertSummary that goes beyond the lesson scope.
- Focus on TRANSFER — connecting this concept to other domains.""",
    }

    def build(
        self,
        allowed_components: list[str],
        scaffold_level: int,
        p_mastery: float,
        concept_name: Optional[str] = None,
        misconception: Optional[dict] = None,
    ) -> str:
        """
        Build the complete system prompt for Claude GenUI generation.
        This prompt enforces the BKT-gated component catalog.
        """

        # 1. Role and output format
        prompt = """You are EduForge's intelligent tutoring UI generator. Your job is to create educational visualizations as structure

OUTPUT FORMAT:
You MUST output ONLY a valid JSON array of component objects. No markdown, no explanation, no wrapper.
Each object must have "component" (string) and "props" (object matching the schema below).
Output 1-3 components per response. Order them by pedagogical priority (most important first).

Example output:
[
  {"component": "StepByStep", "props": {"concept": "...", "steps": [...], "summary": "..."}},
  {"component": "FormulaCard", "props": {"formula": "...", "variables": [...]}}
]

"""

        # 2. BKT state context
        prompt += f"""STUDENT STATE:
- P(mastery) = {p_mastery:.2f}
- Scaffold Level: {scaffold_level}/4
- Classification: {['Novice', 'Developing', 'Approaching', 'Proficient', 'Mastered'][scaffold_level]}

"""

        # 3. Allowed components with schemas
        prompt += "ALLOWED COMPONENTS (you may ONLY use these — any other component will be rejected):\n\n"
        for comp_name in allowed_components:
            if comp_name in self.COMPONENT_SCHEMAS:
                prompt += f"### {comp_name}\n{self.COMPONENT_SCHEMAS[comp_name]}\n\n"

        prompt += f"FORBIDDEN COMPONENTS: {', '.join(set(self.COMPONENT_SCHEMAS.keys()) - set(allowed_components))}\n"
        prompt += "If you output a forbidden component, it will be silently dropped and the student sees nothing.\n\n"

        # 4. Pedagogical behavior rules
        prompt += f"PEDAGOGICAL RULES:\n{self.PEDAGOGICAL_RULES.get(scaffold_level, self.PEDAGOGICAL_RULES[2])}\n\n"

        # 5. Misconception handling (if detected)
        if misconception:
            prompt += f"""MISCONCEPTION DETECTED:
Type: {misconception.get('type', 'unknown')}
Description: {misconception.get('explanation', '')}
INSTRUCTION: Address this misconception directly. The student has gotten 3+ consecutive wrong answers
related to this specific misunderstanding. Use the lowest-level scaffold component available
(StepByStep or HintCard) to explicitly correct this misconception before continuing.
Frame it positively: "A common way to think about this is X, but actually Y because Z."

"""

        # 6. Hard constraints
```

```
        prompt += """HARD CONSTRAINTS:
1. Output ONLY valid JSON array. No markdown fences, no explanation text.
2. Use ONLY allowed components. Forbidden components = empty response.
3. Every string value must be educationally accurate. Never fabricate facts.
4. Keep individual text fields under 200 words.
5. LaTeX formulas use $...$ for inline, $$...$$ for display.
6. NEVER mention P(mastery), BKT, scaffold levels, or system internals to the student.
7. NEVER say "As an AI" or break character — you ARE the concept visualization.
"""

        return prompt

    def build_tutor_prompt(
        self,
        concept_name: str,
        scaffold_level: int,
        p_mastery: float,
        allowed_components: list[str],
    ) -> str:
        """Build system prompt for the AI Tutor Chat (conversational, not GenUI JSON)."""

        prompt = f"""You are a warm, expert tutor helping a student understand: {concept_name}

STUDENT MASTERY: {p_mastery:.2f} (Level {scaffold_level}/4: {['Novice','Developing','Approaching','Proficient','Mastered'][scaffold_lev

"""
        prompt += self.PEDAGOGICAL_RULES.get(scaffold_level, self.PEDAGOGICAL_RULES[2])
        prompt += """

CONVERSATION RULES:
- Keep responses concise: 3-5 sentences unless a longer explanation is needed
- Be encouraging and supportive at ALL times
- If the student is confused, reframe the concept from a completely different angle
- Use concrete examples before abstract definitions
- For scaffold 4: use Socratic method — ask leading questions, don't give answers
- NEVER mention P(mastery), BKT, scaffold levels, or any system internals
- NEVER say "As an AI" — you are their personal tutor
- If asked off-topic questions, gently redirect to the concept at hand
"""
        return prompt
```

20.2.2 GenUI Validation Layer (Client-Side)

```
// apps/web/src/components/genui/validator.ts
/**
 * Client-side validation: ensures the LLM's JSON output
 * only contains allowed components for the current scaffold level.
 *
 * This is the SECOND line of defense (after the system prompt).
 * Even if the LLM hallucinates a forbidden component, we strip it here.
 */

import { GATED_LIBRARIES } from './library';

interface GenUIComponentJSON {
  component: string;
  props: Record<string, unknown>;
}

export function validateAndFilterGenUI(
  rawJSON: GenUIComponentJSON[],
  scaffoldLevel: number
): GenUIComponentJSON[] {
  const allowed = new Set(
    Object.keys(GATED_LIBRARIES[scaffoldLevel]?.components ?? {})
      .map(c => c)
  );

  // Map scaffold level to allowed component names
  const ALLOWED_MAP: Record<number, string[]> = {
    0: ['StepByStep', 'HintCard', 'FormulaCard', 'AnalogyCard'],
    1: ['StepByStep', 'HintCard', 'FormulaCard', 'ConceptDiagram'],
    2: ['ConceptDiagram', 'FormulaCard', 'HintCard', 'PracticeExercise'],
    3: ['ConceptDiagram', 'PracticeExercise', 'ProofWalkthrough'],
    4: ['ConceptDiagram', 'ExpertSummary', 'ProofWalkthrough', 'PracticeExercise'],
  };

  const allowedNames = new Set(ALLOWED_MAP[scaffoldLevel] ?? ALLOWED_MAP[2]);

  const filtered = rawJSON.filter(item => {
    if (!item.component || !item.props) return false;
    if (!allowedNames.has(item.component)) {
      console.warn(
        `[GenUI Validator] Stripped forbidden component "${item.component}" ` +
        `at scaffold level ${scaffoldLevel}. Allowed: ${[...allowedNames].join(', ')}`
      );
      return false;
    }
    return true;
  });

  if (filtered.length === 0) {
    console.error('[GenUI Validator] All components were filtered out! Returning fallback.');
    // Return a safe fallback for the current level
    return scaffoldLevel <= 1
      ? [{ component: 'HintCard', props: { hint_level: 'gentle', hint_text: 'Let me help you understand this concept. Try asking a spec
      : [{ component: 'ConceptDiagram', props: { title: 'Concept Overview', diagram_type: 'hierarchy', elements: [{ id: '1', label: 'Lo
  }

  return filtered;
}
```

20.2.3 Vercel AI SDK — GenUI Streaming Integration (Next.js)

```
// apps/web/src/app/api/genui/stream/route.ts
/**
 * Next.js API Route for GenUI streaming using Vercel AI SDK's streamUI pattern.
 *
 * Architecture:
 * 1. Client calls this route with studentId + conceptId
 * 2. Route fetches BKT state from BKT service
 * 3. Route calls GenUI service which streams from Claude
 * 4. JSON components are streamed back via SSE
 * 5. Client-side renderer maps JSON → React components
 */

import { NextRequest } from 'next/server';

export const runtime = 'edge';

export async function POST(req: NextRequest) {
  const { conceptId, subtopicId, lessonId, studentId, type } = await req.json();

  // 1. Get BKT state + scaffold decision in parallel
  const [bktRes, conceptRes] = await Promise.all([
    fetch(`${process.env.BKT_SERVICE_URL}/state?studentId=${studentId}&conceptId=${conceptId}`),
    fetch(`${process.env.API_SERVICE_URL}/lessons/${lessonId}/subtopics/${subtopicId}`),
  ]);

  const bktState = await bktRes.json();
  const conceptData = await conceptRes.json();

  // 2. Get scaffold decision
  const scaffoldRes = await fetch(
    `${process.env.BKT_SERVICE_URL}/scaffold?p_mastery=${bktState.pMastery}`
  );
  const scaffold = await scaffoldRes.json();

  // 3. Stream from GenUI service
  const genUIRes = await fetch(`${process.env.GENUI_SERVICE_URL}/stream/${type}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      concept: conceptData.title,
      concept_content: conceptData.description,
      bkt_state: bktState,
      scaffold_decision: scaffold,
      student_name: 'Student',
    }),
  });

  // 4. Pass through stream
  return new Response(genUIRes.body, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive',
    },
  });
}
```

```
// apps/web/src/hooks/useGenUI.ts
/**
 * Client-side hook that consumes the GenUI SSE stream,
 * parses the JSON, validates against scaffold level,
 * and returns renderable component data.
 */

import { useState, useCallback, useRef } from 'react';
import { validateAndFilterGenUI } from '@/components/genui/validator';
import { useBKTStore } from '@/stores/bktStore';

interface GenUIComponent {
  component: string;
  props: Record<string, unknown>;
}

interface UseGenUIReturn {
  components: GenUIComponent[];
  isStreaming: boolean;
  error: string | null;
  generate: (conceptId: string, subtopicId: string, lessonId: string) => Promise<void>;
}

export function useGenUI(studentId: string): UseGenUIReturn {
  const [components, setComponents] = useState<GenUIComponent[]>([]);
  const [isStreaming, setIsStreaming] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const scaffoldLevel = useBKTStore(s => s.scaffoldLevel);
  const abortRef = useRef<AbortController | null>(null);

  const generate = useCallback(async (
    conceptId: string, subtopicId: string, lessonId: string
  ) => {
    // Cancel any in-flight request
    abortRef.current?.abort();
    abortRef.current = new AbortController();

    setIsStreaming(true);
    setError(null);
    setComponents([]);

    try {
      const res = await fetch('/api/genui/stream', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          conceptId, subtopicId, lessonId, studentId, type: 'visualize',
        }),
        signal: abortRef.current.signal,
      });

      if (!res.ok) throw new Error(`GenUI stream failed: ${res.status}`);
      if (!res.body) throw new Error('No stream body');

      const reader = res.body.getReader();
      const decoder = new TextDecoder();
      let accumulated = '';

      while (true) {
        const { done, value } = await reader.read();
        if (done) break;

        accumulated += decoder.decode(value, { stream: true });

        // Try to parse accumulated JSON (may be partial during streaming)
        try {
          const parsed = JSON.parse(accumulated);
          if (Array.isArray(parsed)) {
            const validated = validateAndFilterGenUI(parsed, scaffoldLevel);
            setComponents(validated);
          }
        } catch {
          // JSON not yet complete — continue accumulating
        }
      }

      // Final parse attempt
      try {
        const finalParsed = JSON.parse(accumulated);
```

```
        if (Array.isArray(finalParsed)) {
          const validated = validateAndFilterGenUI(finalParsed, scaffoldLevel);
          setComponents(validated);
        }
      } catch (e) {
        setError('Failed to parse GenUI response');
      }

    } catch (e: any) {
      if (e.name !== 'AbortError') {
        setError(e.message);
      }
    } finally {
      setIsStreaming(false);
    }
  }, [studentId, scaffoldLevel]);

  return { components, isStreaming, error, generate };
}
```

20.3 End-to-End Flow: MCQ Answer → BKT Update → GenUI Re-render

```
Student answers MCQ (correct/incorrect)
    │
    ▼
AdaptiveMCQ.tsx → POST /bkt/update
    │
    ▼
BKT Service:
  1. PyBKTEngine.update() → Roster.update_state() → new P(mastery)
  2. ScaffoldResolver.resolve(p_mastery) → scaffold level + allowed components
  3. MisconceptionDetector (if 3+ consecutive wrong)
  4. Save to Firestore → triggers onSnapshot on client
    │
    ▼
Returns: { p_mastery_after, scaffold_level, allowed_components, misconception }
    │
    ▼
Client receives response:
  1. useBKTStore → updates scaffoldLevel + allowedComponents
  2. MasteryHUD re-renders with new P(mastery)
  3. If scaffold level CHANGED → useGenUI.generate() is called
  4. GenUI panel streams new components from Claude (constrained by NEW scaffold level)
    │
    ▼
Student sees: updated mastery bar + potentially new UI layout adapted to their level
```

---

21. VS Code Agent Skills & Tooling

> This section tells the developer AI agent which VS Code Copilot agent skills to install and use, and which npm packages to leverage for best practices across the codebase.

21.1 Agent Skills for VS Code Copilot

The developer AI agent (GitHub Copilot in VS Code) should install and use the following skills. The agent can install these from the VS Code marketplace or from npm-based skill repositories.

21.1.1 UI/UX Skills

Skill	Purpose	When to Use
`baseline-ui`	Validates animation durations, typography scale, component accessibility (a11y), layout anti-patterns in Tailwind CSS	Every time building or reviewing a React component with Tailwind classes. Enforce WCAG 2.1 AA compliance.
`find-skills`	Discovers and installs additional agent skills dynamically	When the agent needs a capability not covered by installed skills

Instructions for the agent:

```
When building ANY React component in apps/web/src/components/:
1. Load the baseline-ui skill FIRST
2. Validate all Tailwind utility classes against the skill's rules
3. Ensure all interactive elements have proper aria-* attributes
4. Check color contrast ratios meet WCAG 2.1 AA (4.5:1 for text)
5. Ensure animations use prefers-reduced-motion media query
6. Verify touch targets are at least 44x44px for student mobile use
```

21.1.2 Architecture & Best Practices — npm Packages the Agent Should Know

Frontend (Next.js / React) — Install with `pnpm`:

Package	Purpose	Agent Instruction
`eslint` + `@eslint/js`	Linting	Configure with Next.js recommended rules. Run on every file save.
`prettier`	Code formatting	Use consistent formatting. 2-space indent, single quotes, trailing commas.
`@typescript-eslint/eslint-plugin`	TypeScript-specific linting	Enforce strict mode. No `any` types except in explicitly typed escape hatches.
`eslint-plugin-react-hooks`	Hooks rules enforcement	Prevent stale closures, enforce exhaustive deps. Critical for real-time Firestore listeners.
`eslint-plugin-jsx-a11y`	Accessibility linting	Required for all student-facing components. Students may have disabilities.
`zod`	Runtime schema validation	Validate ALL API responses and GenUI JSON output. Never trust external data.
`next-safe-action`	Type-safe server actions	Use for all server actions in the Next.js app.
`@tanstack/react-query`	Server state management	Use for API calls to backend services. Configure staleTime per endpoint.
`zustand`	Client state management	Already specified in Section 13. Use for BKT state store.
`framer-motion`	Animations	Use for MCQ transitions, mastery bar updates, and GenUI component entrance animations.
`react-hook-form`	Form handling	Use for teacher lesson creation form- and student enrollment forms.
`reactflow`	Graph visualization	Teacher concept graph editor (Section 5.4).
`d3`	Data visualization	Mastery HUD radar chart, analytics heatmap.
`lucide-react`	Icons	Consistent icon set across the app.
`tailwind-merge`	Class merging	Use `cn()` helper with `clsx` + `tailwind-merge` for conditional class application.
`clsx`	Conditional classes	Always use with `tailwind-merge` via a `cn()` utility function.

Backend (Python / FastAPI) — Install with `pip`:

Package	Purpose	Agent Instruction
`fastapi`	API framework	Use for all backend services. Enable OpenAPI docs at `/docs`.
`uvicorn[standard]`	ASGI server	Production server. Use `--workers 2` for Cloud Run.
`pydantic`	Data validation	Use Pydantic v2 models for ALL request/response schemas.
`pyBKT`	Bayesian Knowledge Tracing	Use Roster API for online updates, Model.fit() for batch parameter fitting.
`anthropic`	Claude API client	Use streaming API (`messages.stream()`) for GenUI generation.
`google-generativeai`	Gemini API client	Use for ingestion pipeline (topic hierarchy, MCQ generation).
`google-cloud-firestore`	Firestore client	Use `AsyncClient` for all Firestore operations.
`google-cloud-storage`	GCS client	Use for PPT upload/download, image assets.
`google-cloud-documentai`	Document AI client	Use Layout Parser for PPTX structure extraction.
`google-cloud-pubsub`	Pub/Sub client	Use for async ingestion triggers and completion notifications.
`python-pptx`	PPTX extraction	Use for slide-level text, image, and notes extraction.
`structlog`	Structured logging (Python)	Use instead of stdlib `logging` for JSON-structured logs in Cloud Run.
`httpx`	Async HTTP client	Use for service-to-service calls within the backend.
`tenacity`	Retry logic	Use for Gemini/Claude API calls with exponential backoff.

21.1.3 Agent Instructions — Coding Standards

```
## For the VS Code AI Agent — Coding Standards to Follow

### TypeScript (Frontend)
- Use `strict: true` in tsconfig.json
- Prefer `interface` over `type` for object shapes
- Use `as const` for enum-like objects
- Never use `any` — use `unknown` + type guards instead
- All components must be functional components with explicit return types
- Use `React.FC` sparingly — prefer `function ComponentName(props: Props): JSX.Element`
- Server Components by default; add 'use client' only when hooks/interactivity needed
- Always handle loading, error, and empty states in UI components

### Python (Backend)
- Use Python 3.11+ features (match statements, ExceptionGroup, etc.)
- All functions must have type annotations
- Use `async def` for ANY function that does I/O (Firestore, HTTP, file)
- Use Pydantic v2 `BaseModel` for all data shapes
- Use `structlog` for all logging (see Section 22)
- Raise `HTTPException` with meaningful status codes and messages
- Never catch bare `Exception` — catch specific exception types

### Git Conventions
- Commit messages: `type(scope): description` (e.g., `feat(bkt): add pyBKT Roster integration`)
- Types: feat, fix, refactor, test, docs, chore, perf
- Scopes: bkt, genui, ingestion, api, web, infra, shared

### Testing
- Unit tests: colocate with source in `__tests__/` or `tests/` directories
- Use pytest for Python, Jest + React Testing Library for TypeScript
- BKT engine must have >90% test coverage
- Every API endpoint must have at least one integration test
```

21.2 Recommended `.vscode/settings.json`

```
{
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit",
    "source.organizeImports": "explicit"
  },
  "typescript.preferences.importModuleSpecifier": "non-relative",
  "typescript.tsdk": "node_modules/typescript/lib",
  "python.defaultInterpreterPath": ".venv/bin/python",
  "python.analysis.typeCheckingMode": "basic",
  "[python]": {
    "editor.defaultFormatter": "ms-python.black-formatter",
    "editor.formatOnSave": true
  },
  "files.associations": {
    "*.css": "tailwindcss"
  },
  "tailwindCSS.experimental.classRegex": [
    ["cn\\(([^)]*)\\)", "'([^']*)'"],
    ["clsx\\(([^)]*)\\)", "'([^']*)'"]
  ]
}
```

21.3 Recommended `.vscode/extensions.json`

```
{
  "recommendations": [
    "esbenp.prettier-vscode",
    "dbaeumer.vscode-eslint",
    "bradlc.vscode-tailwindcss",
    "ms-python.python",
    "ms-python.black-formatter",
    "ms-python.pylance",
    "github.copilot",
    "github.copilot-chat",
    "ms-azuretools.vscode-docker",
    "usernamehw.errorlens",
    "gruntfuggly.todo-tree"
  ]
}
```

---

22. Structured Logging & Chalk Strategy

> This section defines the logging architecture for BOTH the Node.js frontend and Python backend. Every log should be structured, colored in development, and JSON in production.

22.1 Philosophy: Why Structured Logging Matters

EduForge has 4 backend services + 1 frontend. Without structured logging:

- You can't search logs by `studentId`, `lessonId`, or `conceptId` in Cloud Logging
- You can't correlate a BKT update with the GenUI call it triggered
- You can't measure p95 latency of the ingestion pipeline
- Competition judges can't see the system working in real-time during the demo

Rule: Every log message must include structured context. Every service must use a consistent log format.

22.2 Frontend Logging — chalk + Custom Logger (Next.js)

> Use `chalk` v5 (ESM) for colored development logs. In production, use `pino` for JSON structured logs.

```typescript
// apps/web/src/lib/logger.ts
/**
 * EduForge Frontend Logger
 *
 * Development: chalk-colored output with emoji prefixes
 * Production: pino JSON logs for Cloud Logging
 *
 * Usage:
 *   import { logger } from '@/lib/logger';
 *   logger.info('BKT state updated', { studentId, conceptId, pMastery: 0.45 });
 *   logger.bkt('Scaffold level changed', { from: 1, to: 2, studentId });
 *   logger.genui('Streaming started', { conceptId, scaffoldLevel: 2 });
 */

import chalk from 'chalk';

type LogLevel = 'debug' | 'info' | 'warn' | 'error';

interface LogContext {
  [key: string]: string | number | boolean | null | undefined;
}

// Service-specific log prefixes with chalk colors
const PREFIXES = {
  bkt:       chalk.magenta.bold('[BKT]'),
  genui:     chalk.cyan.bold('[GenUI]'),
  ingestion: chalk.yellow.bold('[Ingest]'),
  auth:      chalk.green.bold('[Auth]'),
  api:       chalk.blue.bold('[API]'),
  firestore: chalk.gray.bold('[Firestore]'),
  general:   chalk.white.bold('[App]'),
} as const;

const LEVEL_COLORS = {
  debug: chalk.gray,
  info:  chalk.blue,
  warn:  chalk.yellow,
  error: chalk.red.bold,
} as const;

const LEVEL_ICONS = {
  debug: '🔍',
  info:  '📘',
  warn:  '⚠️',
  error: '❌',
} as const;

const isDev = process.env.NODE_ENV !== 'production';

function formatContext(ctx: LogContext): string {
  if (Object.keys(ctx).length === 0) return '';

  if (isDev) {
    // Pretty-print context in development with colors
    const pairs = Object.entries(ctx)
      .filter(([, v]) => v !== undefined)
      .map(([k, v]) => `${chalk.dim(k)}=${chalk.white(String(v))}`);
    return ` ${chalk.dim('|')} ${pairs.join(chalk.dim(', '))}`;
  }

  // JSON in production
  return ` ${JSON.stringify(ctx)}`;
}

function log(
  level: LogLevel,
  prefix: keyof typeof PREFIXES,
  message: string,
  context: LogContext = {}
) {
  const timestamp = new Date().toISOString();

  if (isDev) {
    const icon = LEVEL_ICONS[level];
    const coloredLevel = LEVEL_COLORS[level](level.toUpperCase().padEnd(5));
    const coloredPrefix = PREFIXES[prefix];
    const coloredMsg = level === 'error' ? chalk.red(message) : message;
    const ctxStr = formatContext(context);
```

```
    const output = `${chalk.dim(timestamp)} ${icon} ${coloredLevel} ${coloredPrefix} ${coloredMsg}${ctxStr}`;

    if (level === 'error') console.error(output);
    else if (level === 'warn') console.warn(output);
    else console.log(output);
  } else {
    // Production: JSON structured log for Cloud Logging
    const logObj = {
      severity: level.toUpperCase(),
      timestamp,
      service: 'eduforge-web',
      component: prefix,
      message,
      ...context,
    };

    if (level === 'error') console.error(JSON.stringify(logObj));
    else console.log(JSON.stringify(logObj));
  }
}

export const logger = {
  // General logging
  debug: (msg: string, ctx?: LogContext) => log('debug', 'general', msg, ctx),
  info:  (msg: string, ctx?: LogContext) => log('info',  'general', msg, ctx),
  warn:  (msg: string, ctx?: LogContext) => log('warn',  'general', msg, ctx),
  error: (msg: string, ctx?: LogContext) => log('error', 'general', msg, ctx),

  // Service-specific loggers (colored prefixes)
  bkt:       (msg: string, ctx?: LogContext) => log('info', 'bkt', msg, ctx),
  genui:     (msg: string, ctx?: LogContext) => log('info', 'genui', msg, ctx),
  ingestion: (msg: string, ctx?: LogContext) => log('info', 'ingestion', msg, ctx),
  auth:      (msg: string, ctx?: LogContext) => log('info', 'auth', msg, ctx),
  api:       (msg: string, ctx?: LogContext) => log('info', 'api', msg, ctx),
  firestore: (msg: string, ctx?: LogContext) => log('debug', 'firestore', msg, ctx),
};
```

22.3 Backend Logging — structlog (Python Services)

> Use `structlog` for all Python backend services. JSON in production, colored in dev.

```python
# packages/shared-python/logging_config.py
"""
Shared logging configuration for ALL Python backend services.
Import this once in each service's main.py.

Usage:
    from logging_config import setup_logging, get_logger
    setup_logging()
    logger = get_logger('bkt.engine')
    logger.info('BKT update', student_id='s1', concept_id='c1', p_mastery=0.45)
"""

import structlog
import logging
import os
import sys

def setup_logging():
    """Call once at service startup."""
    is_dev = os.environ.get('ENV', 'production') != 'production'

    # Shared processors for all log entries
    shared_processors = [
        structlog.contextvars.merge_contextvars,
        structlog.stdlib.add_log_level,
        structlog.stdlib.add_logger_name,
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.StackInfoRenderer(),
        structlog.processors.UnicodeDecoder(),
    ]

    if is_dev:
        # Development: colored, human-readable output
        renderer = structlog.dev.ConsoleRenderer(
            colors=True,
            pad_event=40,
        )
    else:
        # Production: JSON for Cloud Logging
        # Cloud Logging expects 'severity' instead of 'level'
        shared_processors.append(_rename_level_to_severity)
        renderer = structlog.processors.JSONRenderer()

    structlog.configure(
        processors=shared_processors + [
            structlog.stdlib.ProcessorFormatter.wrap_for_formatter,
        ],
        logger_factory=structlog.stdlib.LoggerFactory(),
        wrapper_class=structlog.stdlib.BoundLogger,
        cache_logger_on_first_use=True,
    )

    formatter = structlog.stdlib.ProcessorFormatter(
        processor=renderer,
        foreign_pre_chain=shared_processors,
    )

    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(formatter)

    root = logging.getLogger()
    root.handlers.clear()
    root.addHandler(handler)
    root.setLevel(logging.DEBUG if is_dev else logging.INFO)

    # Silence noisy libraries
    logging.getLogger('google').setLevel(logging.WARNING)
    logging.getLogger('urllib3').setLevel(logging.WARNING)
    logging.getLogger('httpx').setLevel(logging.WARNING)


def _rename_level_to_severity(logger, method_name, event_dict):
    """Rename 'level' to 'severity' for Google Cloud Logging compatibility."""
    if 'level' in event_dict:
        event_dict['severity'] = event_dict.pop('level').upper()
    return event_dict


def get_logger(name: str):
```

```
    """Get a structured logger bound to a service component name."""
    return structlog.get_logger(name)
```

22.3.1 Service-Specific Usage Examples

```
# apps/bkt-service/main.py
from logging_config import setup_logging, get_logger
setup_logging()

logger = get_logger('bkt-service')

@app.post("/update")
async def update_bkt(request: MCQAnswerRequest):
    logger.info(
        "BKT update started",
        student_id=request.student_id,
        concept_id=request.concept_id,
        is_correct=request.is_correct,
    )

    # ... BKT update logic ...

    logger.info(
        "BKT update complete",
        student_id=request.student_id,
        concept_id=request.concept_id,
        p_mastery_before=p_before,
        p_mastery_after=p_after,
        scaffold_level=scaffold.level,
        mastered=new_state.mastered,
        duration_ms=round(duration * 1000, 2),
    )
```

```
# apps/genui-service/main.py
from logging_config import setup_logging, get_logger
setup_logging()

logger = get_logger('genui-service')

async def stream_visualization(self, concept, bkt_state, scaffold):
    logger.info(
        "GenUI stream started",
        concept=concept,
        scaffold_level=scaffold['level'],
        p_mastery=bkt_state['p_mastery'],
        allowed_components=','.join(scaffold['allowed_components']),
    )

    token_count = 0
    async for token in stream:
        token_count += 1
        yield token

    logger.info(
        "GenUI stream complete",
        concept=concept,
        tokens_generated=token_count,
        scaffold_level=scaffold['level'],
    )
```

```python
# apps/ingestion/main.py
from logging_config import setup_logging, get_logger
setup_logging()

logger = get_logger('ingestion')

async def run_ingestion(lesson_id, gcs_path):
    logger.info("Ingestion started", lesson_id=lesson_id, gcs_path=gcs_path)

    # Step 1
    logger.info("Extracting slides", lesson_id=lesson_id, step="pptx_extract")
    slides = extractor.extract(file_bytes, lesson_id)
    logger.info("Slides extracted", lesson_id=lesson_id, slide_count=len(slides))

    # Step 2
    logger.info("Parsing layout", lesson_id=lesson_id, step="document_ai")
    # ...

    # Step 3
    logger.info("Generating topics", lesson_id=lesson_id, step="gemini_topics")
    subtopics = topic_gen.generate(slide_dicts, lesson_id)
    logger.info("Topics generated", lesson_id=lesson_id, subtopic_count=len(subtopics))

    # Step 4
    logger.info("Generating MCQs", lesson_id=lesson_id, step="gemini_mcqs", subtopic_count=len(subtopics))
    # ... parallel MCQ generation ...
    logger.info("MCQs generated", lesson_id=lesson_id, total_mcqs=len(subtopics) * 15)

    logger.info("Ingestion complete", lesson_id=lesson_id,
                subtopics=len(subtopics), mcqs=len(subtopics)*15,
                duration_seconds=round(time.time() - start, 2))
```

22.4 Where Logs Go — Full Codebase Log Placement Map

Location	What to Log	Level	Context Fields
Auth middleware (all services)	Login/logout, token validation, role checks	`info` / `warn`	`userId`, `role`, `action`
API Gateway routes	Every request + response time	`info`	`method`, `path`, `statusCode`, `durationMs`, `userId`
Lesson upload	Upload start, GCS write, Pub/Sub publish	`info`	`lessonId`, `teacherId`, `fileSize`, `fileName`
Ingestion pipeline	Each step start/end, slide count, subtopic count, MCQ count	`info`	`lessonId`, `step`, `progress`, `duration`
Ingestion errors	Document AI failures, Gemini API errors, parse failures	`error`	`lessonId`, `step`, `errorType`, `errorMessage`
BKT update	Every MCQ answer + BKT state change	`info`	`studentId`, `conceptId`, `isCorrect`, `pMasteryBefore`, `pMasteryAfter`, `scaffoldLevel`
BKT mastery events	When a student masters a concept	`info`	`studentId`, `conceptId`, `subtopicId`, `attempts`, `pMastery`
Scaffold level change	When scaffold level transitions	`info`	`studentId`, `conceptId`, `fromLevel`, `toLevel`
Misconception detection	When 3+ consecutive wrong answers detected	`warn`	`studentId`, `conceptId`, `consecutiveWrong`, `misconceptionType`
GenUI stream start/end	Claude API call initiation and completion	`info`	`conceptId`, `scaffoldLevel`, `allowedComponents`, `tokenCount`, `durationMs`
GenUI validation	Components stripped by validator	`warn`	`strippedComponent`, `scaffoldLevel`, `reason`
Firestore reads/writes	Collection, document ID, operation type	`debug`	`collection`, `docId`, `operation`
API errors	Any unhandled exception in routes	`error`	`path`, `method`, `errorType`, `errorMessage`, `stack`
Rate limiting	When a user hits rate limits	`warn`	`userId`, `endpoint`, `requestCount`, `windowSeconds`

22.5 FastAPI Request Logging Middleware with structlog

```python
# apps/api/src/middleware/request_logger.py
"""
Middleware that logs every HTTP request with structured context.
Adds request_id for correlation across services.
"""

import time
import uuid
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware
import structlog

logger = structlog.get_logger('api.request')

class RequestLoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        request_id = str(uuid.uuid4())[:8]
        start = time.perf_counter()

        # Bind request_id to all logs within this request
        structlog.contextvars.clear_contextvars()
        structlog.contextvars.bind_contextvars(request_id=request_id)

        logger.info(
            "Request started",
            method=request.method,
            path=str(request.url.path),
            query=str(request.url.query) if request.url.query else None,
        )

        try:
            response = await call_next(request)
            duration_ms = round((time.perf_counter() - start) * 1000, 2)

            logger.info(
                "Request completed",
                method=request.method,
                path=str(request.url.path),
                status_code=response.status_code,
                duration_ms=duration_ms,
            )

            response.headers['X-Request-ID'] = request_id
            response.headers['X-Process-Time'] = f"{duration_ms}ms"
            return response

        except Exception as e:
            duration_ms = round((time.perf_counter() - start) * 1000, 2)
            logger.error(
                "Request failed",
                method=request.method,
                path=str(request.url.path),
                error_type=type(e).__name__,
                error_message=str(e),
                duration_ms=duration_ms,
            )
            raise
```

22.6 Package Installation Summary

Frontend (apps/web):

```bash
pnpm add chalk pino pino-pretty
pnpm add -D @types/node
```

Backend (all Python services — add to each requirements.txt):

```
structlog==24.4.0
```

Note: `chalk` v5 is ESM-only which works with Next.js App Router natively. For any CommonJS contexts, use `chalk@4.1.2` instead.

Appendix B: pyBKT Quick Reference

pyBKT API	EduForge Usage	Section
`Model(seed, num_fits)`	Initialize BKT model per concept	20.1.1
`model.coef_ = {...}`	Seed params from Gemini ingestion estimates	20.1.1
`model.fit(data, skills, fixed=True)`	Initialize with seed params	20.1.1
`model.fit(data, skills)`	Re-fit from real student data (nightly)	20.1.2
`model.params()`	Extract fitted P(L0), P(T), P(S), P(G)	20.1.2
`model.crossvalidate(folds=5)`	Evaluate model accuracy	20.1.2
`Roster(students, skills, model)`	Create student tracker per concept	20.1.1
`roster.update_state(skill, student, response)`	HMM forward update after MCQ	20.1.1
`roster.get_mastery_prob(skill, student)`	Get P(mastery) → feed to scaffold resolver	20.1.1
`roster.get_state_type(skill, student)`	`MASTERED` / `UNMASTERED` classification	20.1.1
`roster.get_correct_probs(skill)`	Predict next-correct for all students	20.1.1
`roster.add_student(skill, student)`	Dynamic student enrollment	20.1.1
`roster.remove_student(skill, student)`	Student unenrollment	20.1.1
`roster.reset_state(skill, student)`	Reset to initial prior (re-take lesson)	20.1.1

This document is the single source of truth for the EduForge build. Every section is actionable and code-complete. Start with Section 3 (GCP Setup), then follow the Day-by-Day timeline in Section 18. For deep BKT+GenUI integration, see Section 20. For agent setup, see Section 21.